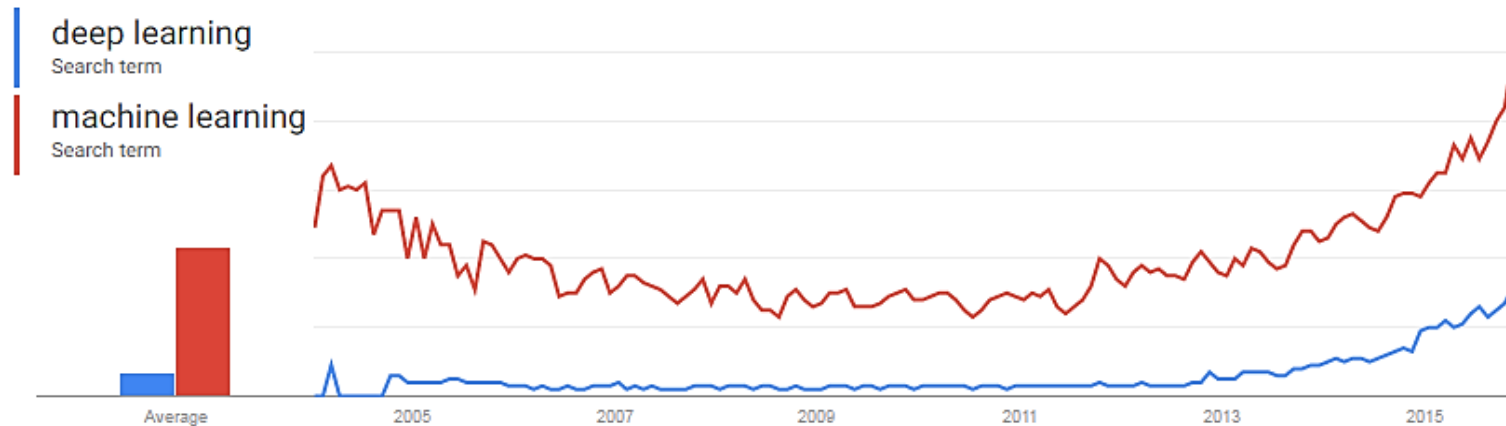
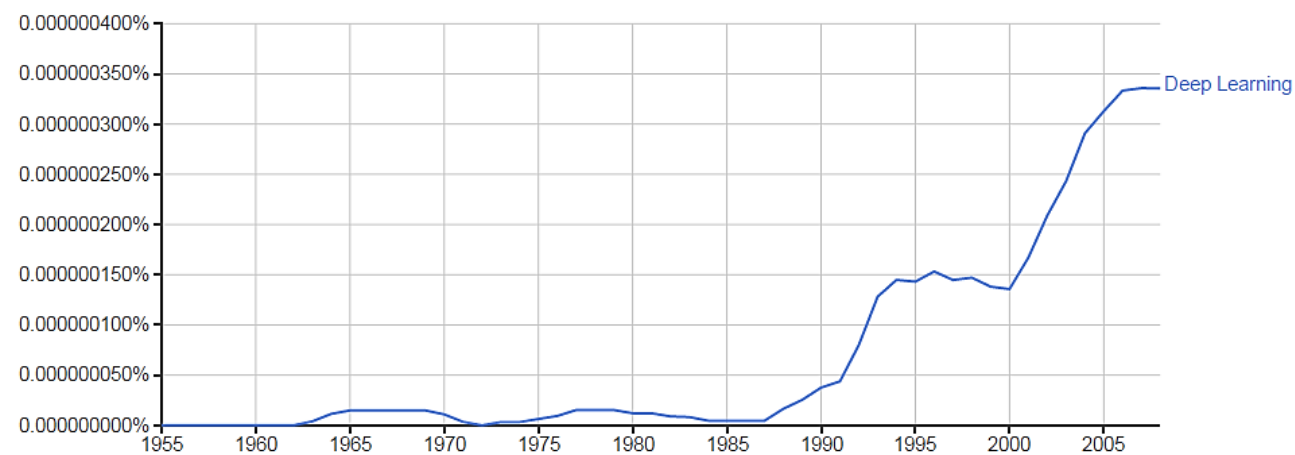




Deep Learning

Interest

Google NGRAM & Google Trends



Hype or Reality?

Quotes

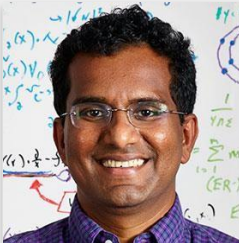


I have worked all my life in Machine Learning, and **I've never seen one algorithm knock over benchmarks like Deep Learning**

– Andrew Ng (Stanford & Baidu)



Deep Learning is an algorithm which has **no theoretical limitations of what it can learn**; the more data you give and the more computational time you provide, the better it is – Geoffrey Hinton (Google)



Human-level artificial intelligence **has the potential to help humanity** thrive more than any invention that has come before it – Dileep George (Co-Founder Vicarious)



For a very long time it will be a **complementary tool** that human scientists and human experts can use to help them with the things that humans are not naturally good – Demis Hassabis (Co-Founder DeepMind)

What is Deep Learning?



Part of the **Data Mining** field of learning representations of data. Exceptional effective at learning patterns.

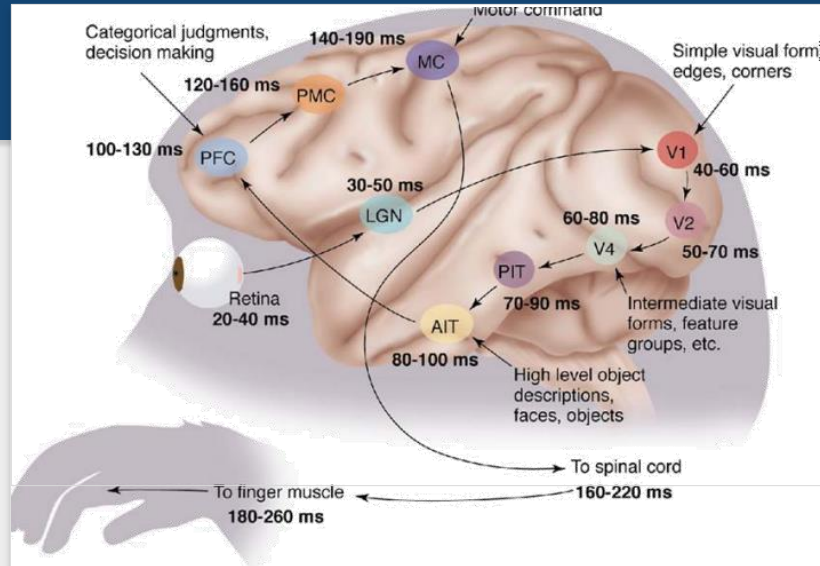


Utilizes learning algorithms that derive meaning out of data by using a **hierarchy** of multiple layers that **mimic the neural networks of our brain**.



If you provide the system tons of information, it begins to understand it and respond in useful ways.

Inspired by the Brain



The first **hierarchy of neurons** that receives information in the visual cortex are sensitive to specific edges while brain regions further down the visual pipeline are sensitive to more complex structures such as faces.



Our brain has lots of neurons connected together and the **strength of the connections** between neurons represents **long term knowledge**.

1

One learning algorithm hypothesis: all significant mental algorithms are learned except for the learning and reward machinery itself.

Why Deep Learning?

Applications



Speech
Recognition



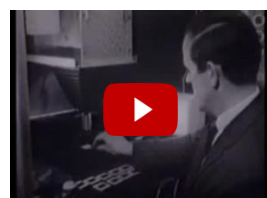
Computer
Vision



Natural Language
Processing

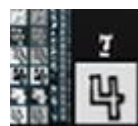
A brief History

A long time ago...



1958 Perceptron

1974 Backpropagation



Convolution Neural Networks for
Handwritten Recognition



1998



Google Brain Project on
16k Cores

2012

1969

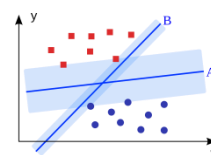
Perceptron criticized



awkward silence (AI Winter)

1995

SVM reigns



2006

Restricted
Boltzmann
Machine



2012

AlexNet wins
ImageNet

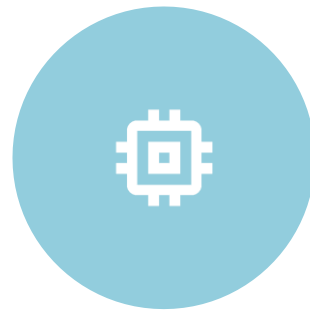
IMAGENET

What changed?

Old wine in new bottles



Big Data
(Digitalization)



Computation
(**Moore's Law, GPUs**)



Algorithmic
Progress

The Big Players

Superstar Researchers



Geoffrey Hinton: University of Toronto & Google



Yann LeCun: New York University & Facebook



Andrew Ng: Stanford & Baidu



Yoshua Bengio: University of Montreal



Jürgen Schmidhuber: Swiss AI Lab & NNAISENSE

The Big Players

Companies

facebook



YAHOO!

Google



IBM



NVIDIA®

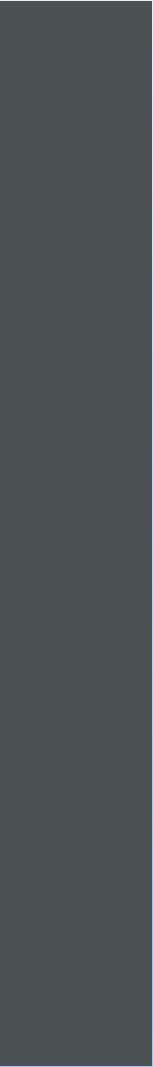
Baidu 百度

The Big Players

Startups



Acquired



Consider These
Problems

Labradoodle or fried chicken



Puppy or bagel



Sheepdog or mop



Chihuahua or muffin



A 4x4 grid of 16 images. The first and third columns show the faces of two different owl species, one with light brown and white plumage, and the other with darker brown and white plumage. The second and fourth columns show the cross-sections of various apple varieties, including green, yellow, and red apples. The owl faces and apple cross-sections are arranged in a way that highlights the visual similarities between them, such as the dark eye sockets of the owls resembling the dark seeds of the apples, and the light-colored facial feathers resembling the pale flesh of the apples. In the bottom-left corner, there is a watermark that reads '@teenybiscuit'.

@teenybiscuit

Parrot or guacamole



Raw chicken or Donald Trump



Deep Learning - Basics

No more feature engineering

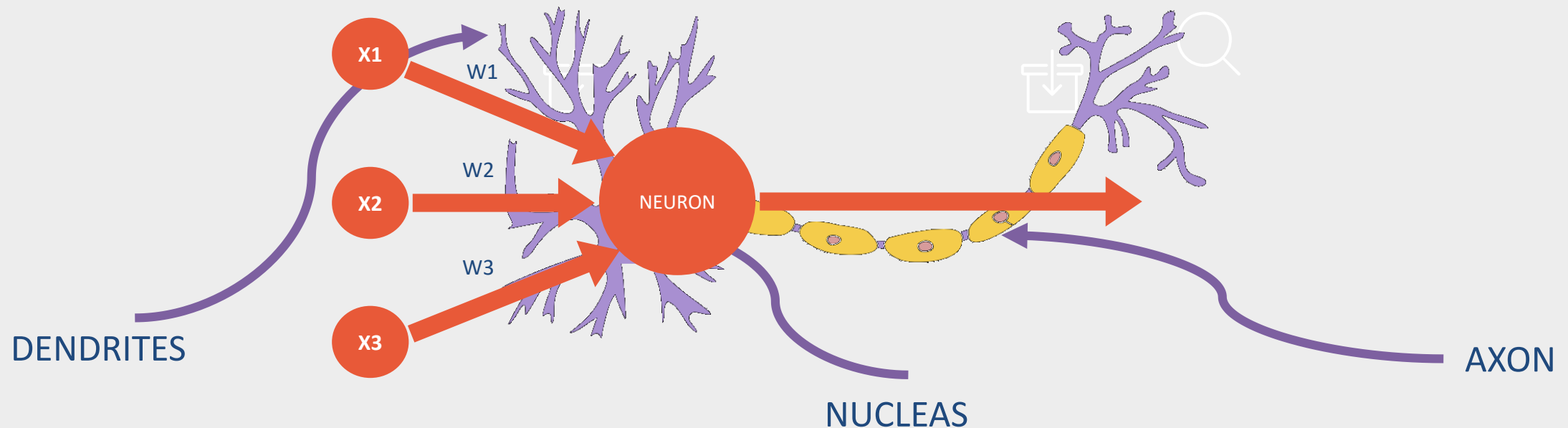


Costs lots of time



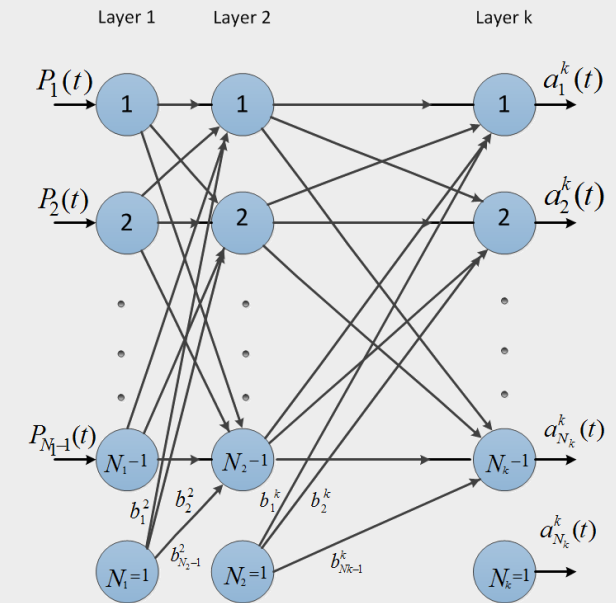
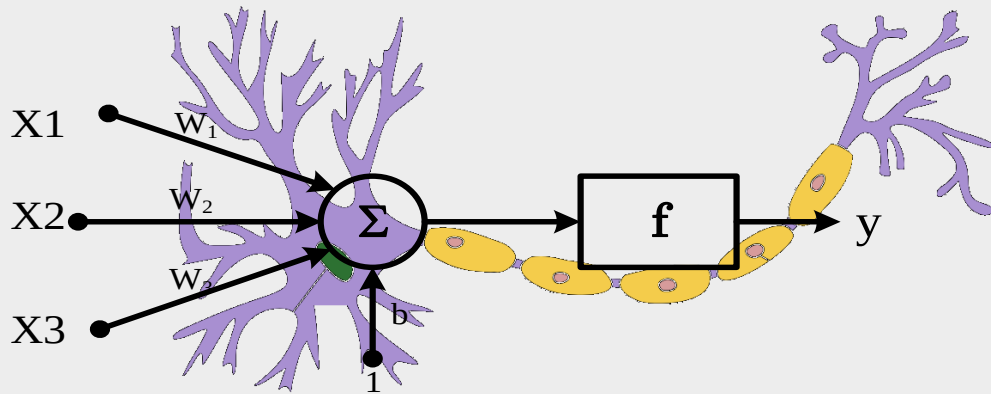
NEURON MATHEMATICAL MODEL

- The neuron collects signals from input channels named dendrites, processes information in its nucleus, and then generates an output in a long thin branch called axon.



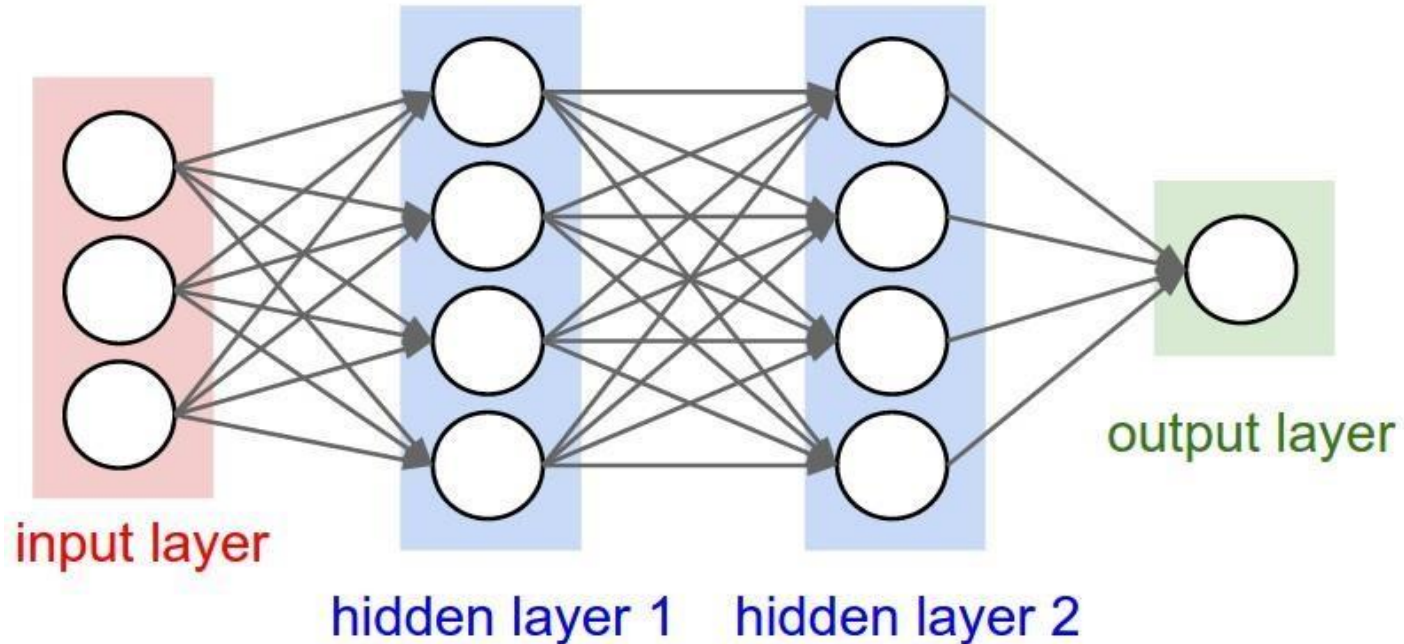
ARTIFICIAL NEURAL NETWORK: INTRODUCTION

- The brain has over 100 billion neurons communicating through electrical and chemical signals.
- Neurons communicate with each other and help us see, think, and generate ideas.
- Human brain learns by creating connections among these neurons.
- ANNs are information processing models inspired by the human brain.



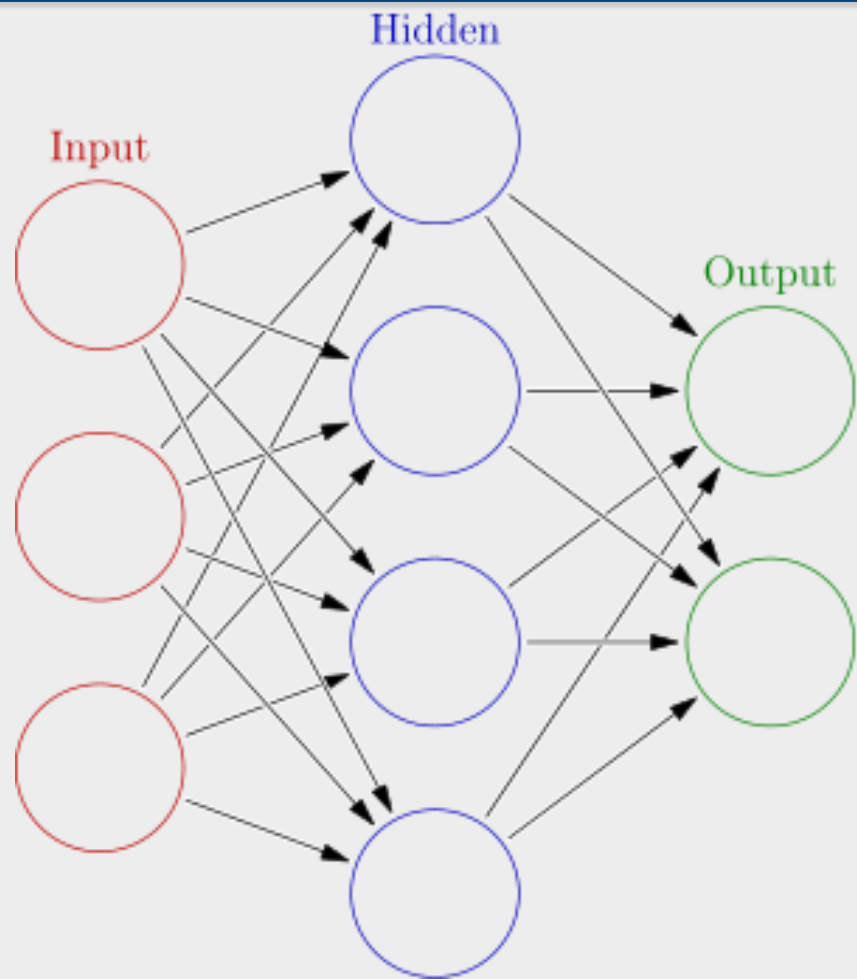
Deep Learning - Basics

Artificial Neural Networks



Consists of one input, one output and multiple fully-connected hidden layers in-between. Each layer is represented as a series of neurons and **progressively extracts higher and higher-level features** of the input until the final layer essentially makes a decision about what the input shows. The more layers the network has, the higher-level features it will learn.

Artificial Neural Network



1. Activation function
2. Weights
3. Cost function
4. Learning algorithm

[Live Demo](#)

HOW DO HUMANS LEARN? FROM EXPERIENCE

- Humans learn from experience (by example)

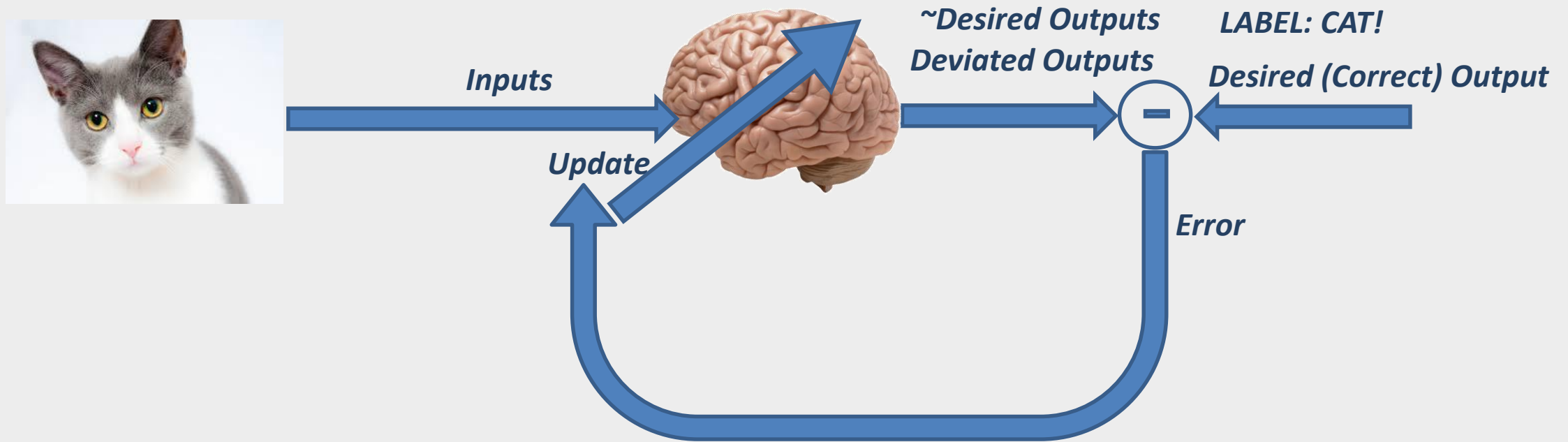
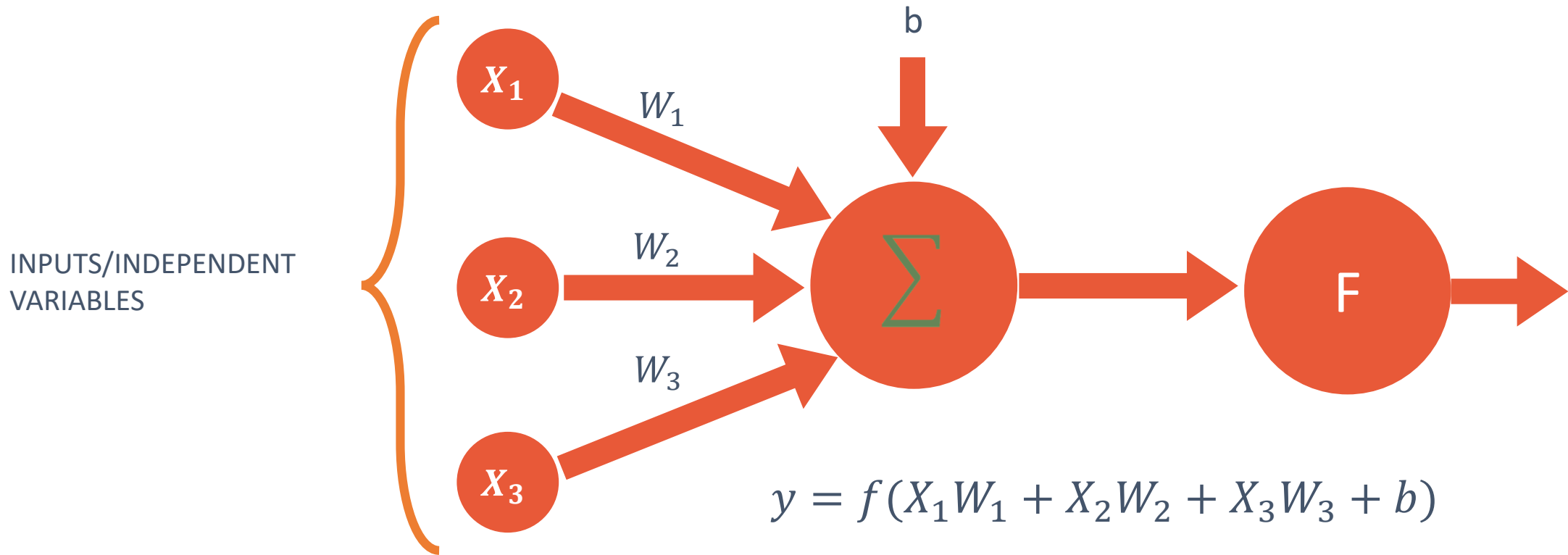


Photo Credit: <https://www.flickr.com/photos/flamephoenix1991/8376271918>

Photo Credit: <https://www.pexels.com/photo/grey-and-white-short-fur-cat-104827/>

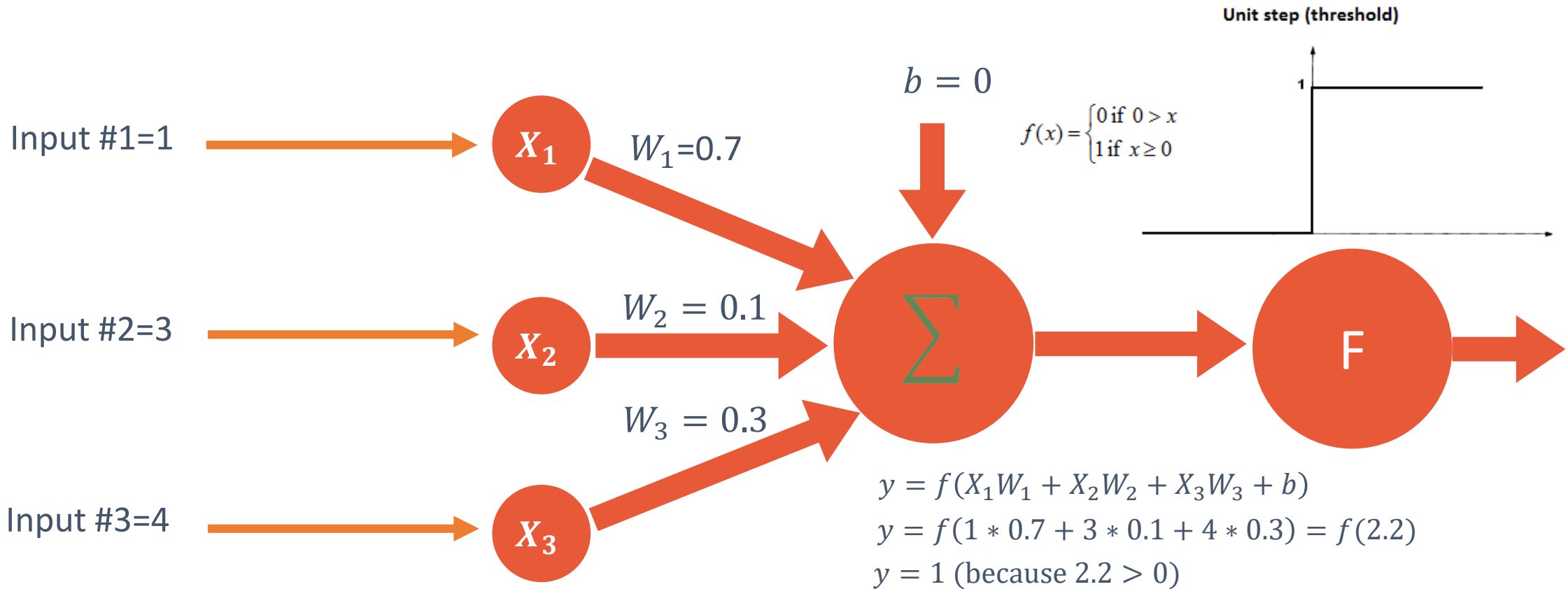
MATHEMATICAL MODEL OF A NEURON

- Bias allows to shift the activation function curve up or down.
- Number of adjustable parameters = 4 (3 weights and 1 bias).
- Activation function “F”.



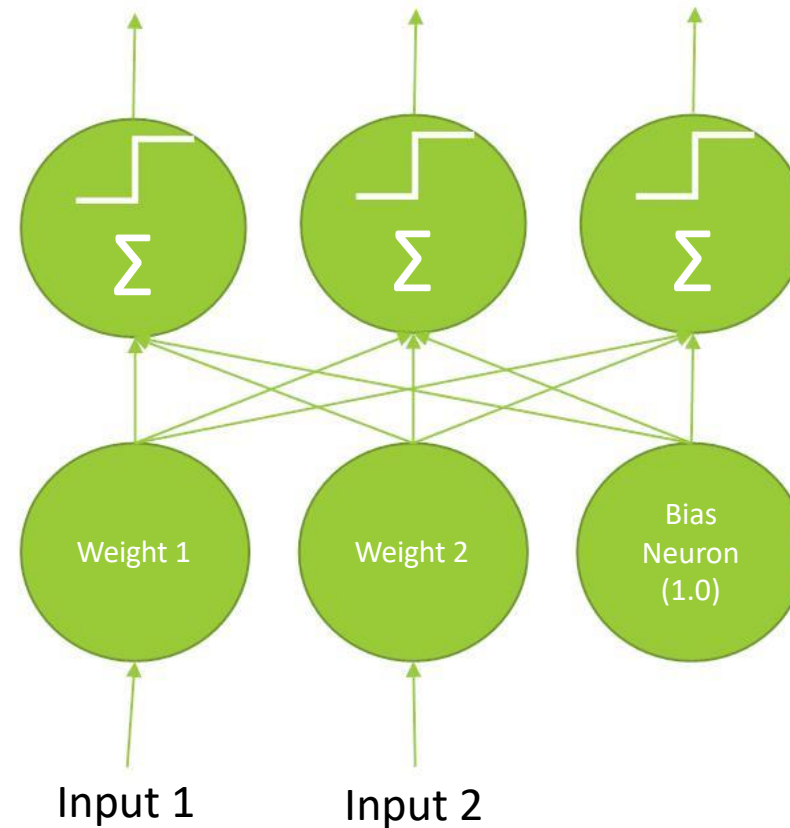
MATHEMATICAL MODEL OF A NEURON

- Let's assume that the activation function is a Unit Step Activation Function.
- The activation functions is used to map the input between (0, 1).



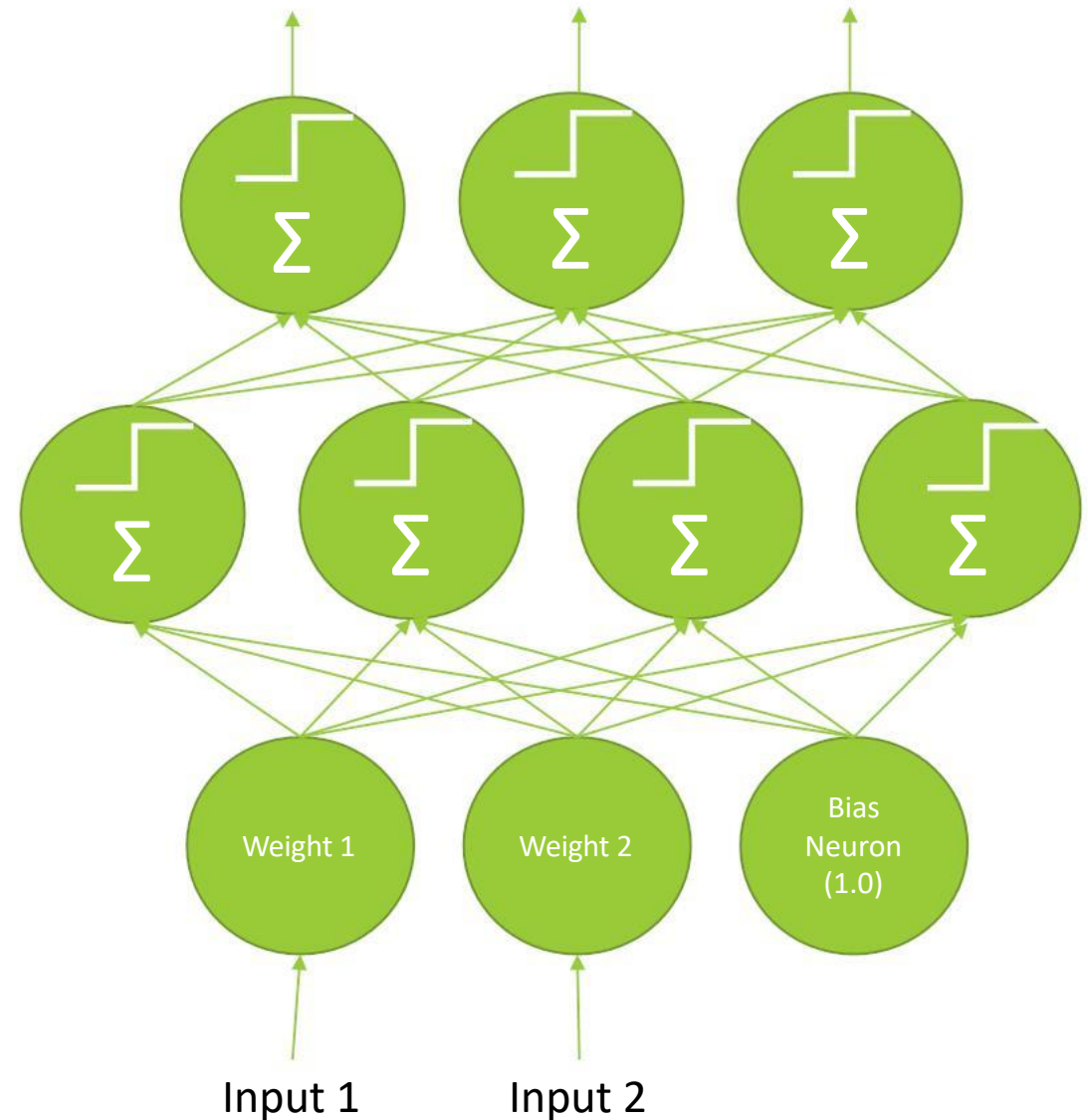
THE PERCEPTRON

- A layer of LTU's
- A perceptron can learn by reinforcing weights that lead to correct behavior during training
- This too has a biological basis (“cells that fire together, wire together”)



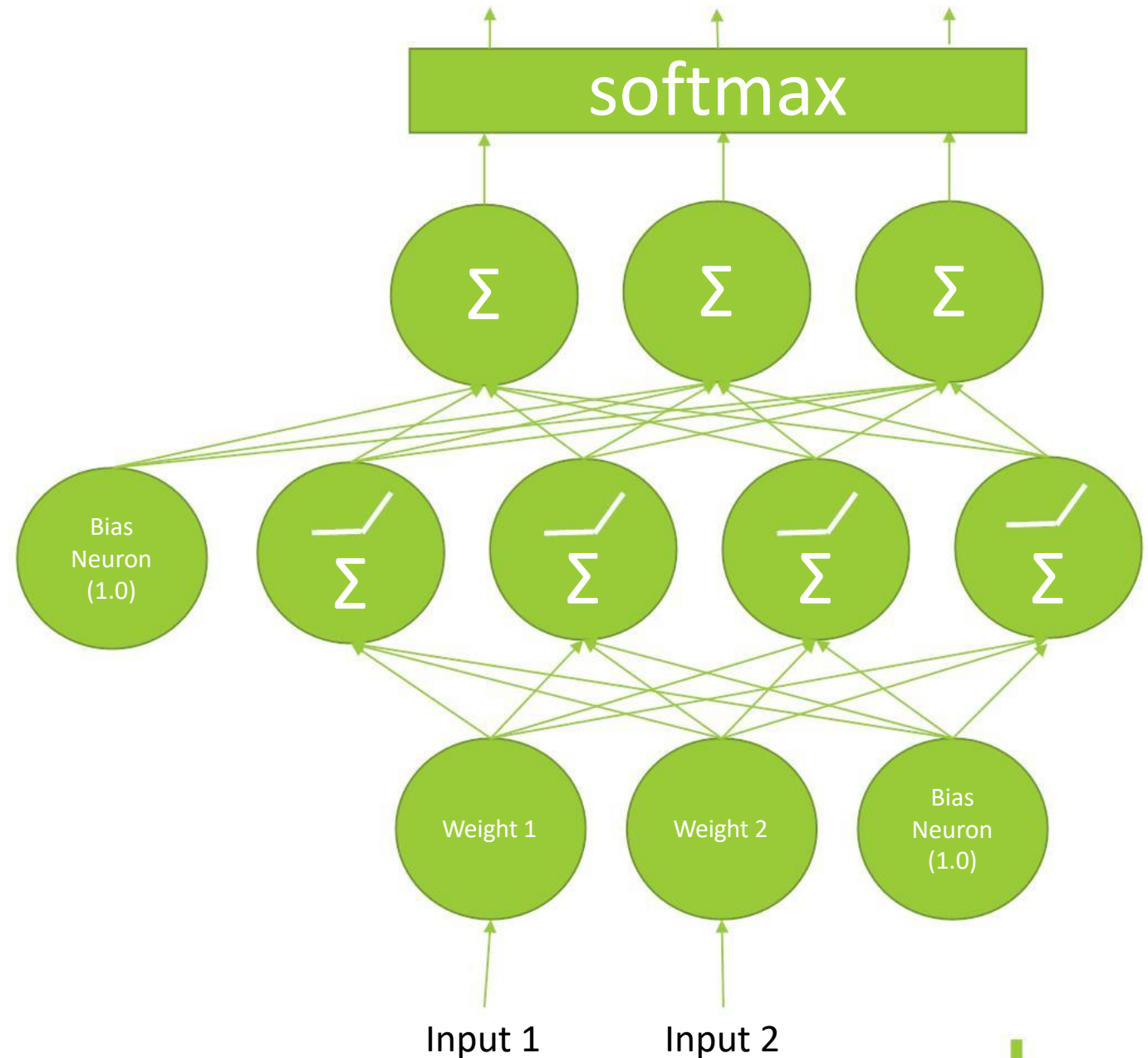
MULTI-LAYER PERCEPTRONS

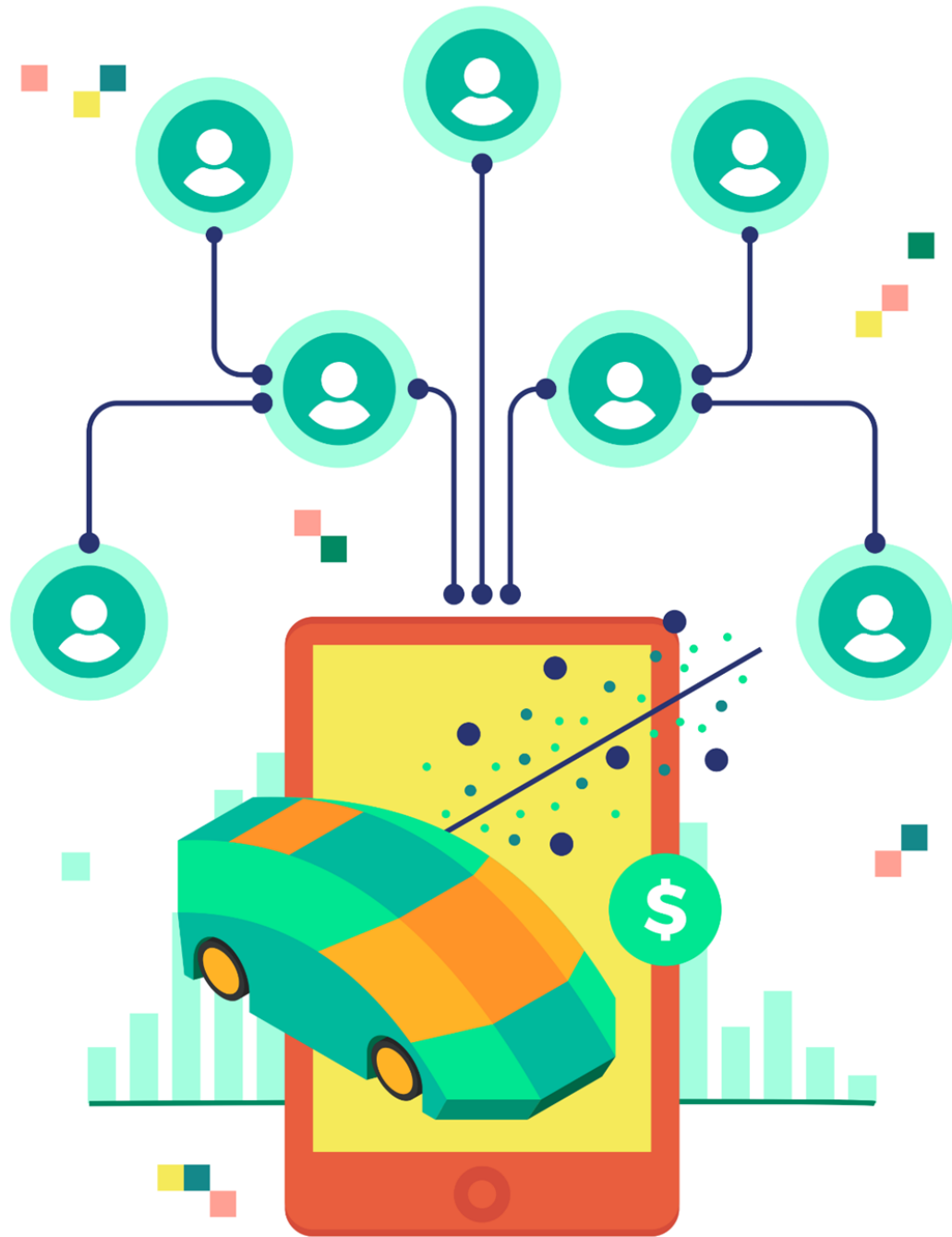
- Addition of “hidden layers”
- This is a Deep Neural Network
- Training them is trickier – but we’ll talk about that.



A MODERN DEEP NEURAL NETWORK

- Replace step activation function with something better
- Apply softmax to the output
- Training using gradient descent

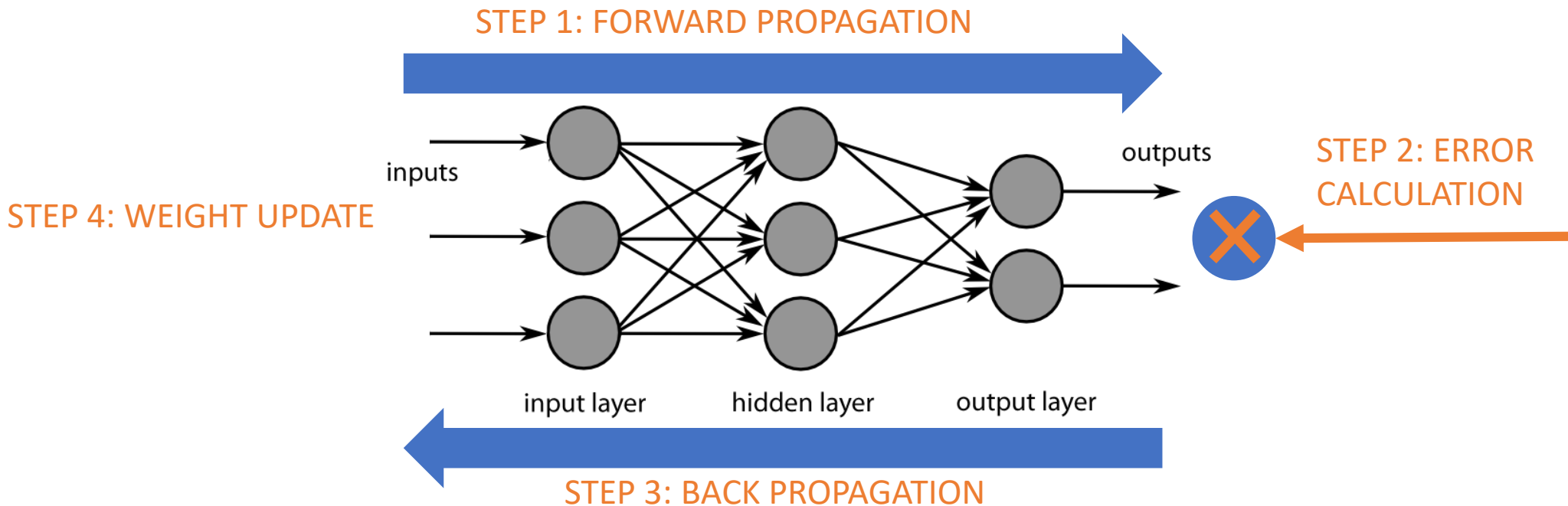




NETWORK TRAINING

NETWORK TRAINING: BACK PROPAGATION

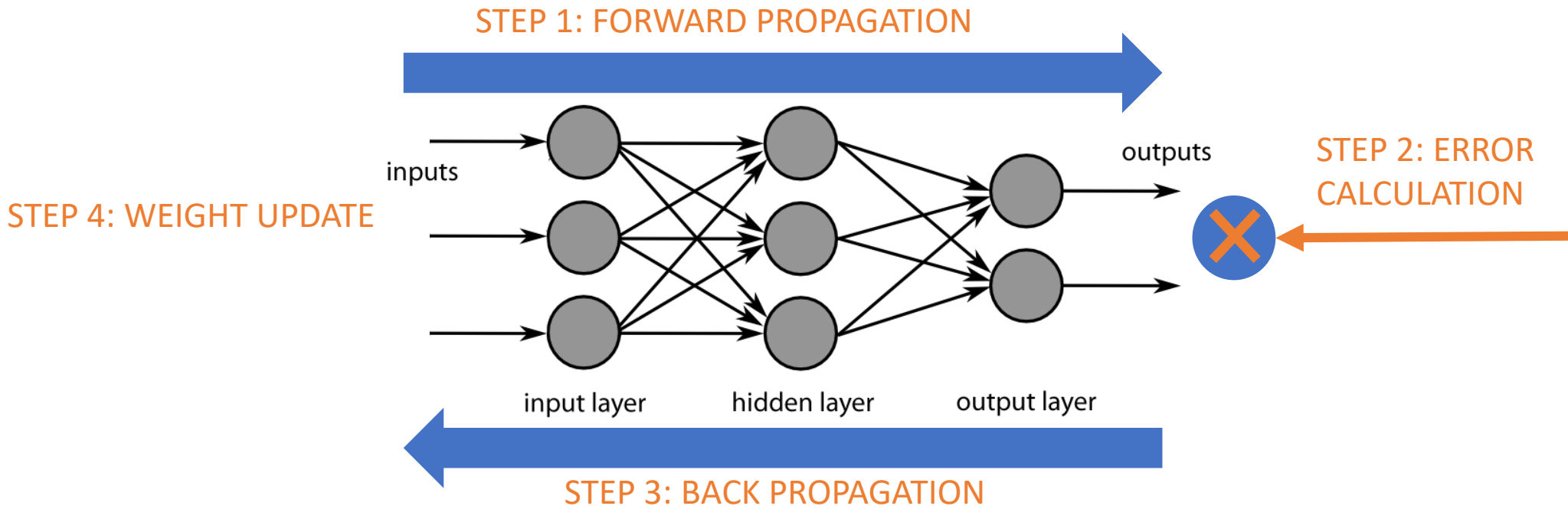
- Backpropagation is a method used to train ANNs by calculating gradient needed to update network weights.
- It is commonly used by the gradient descent optimization algorithm to adjust the weight of neurons by calculating the gradient of the loss function.



NETWORK TRAINING: BACK PROPAGATION

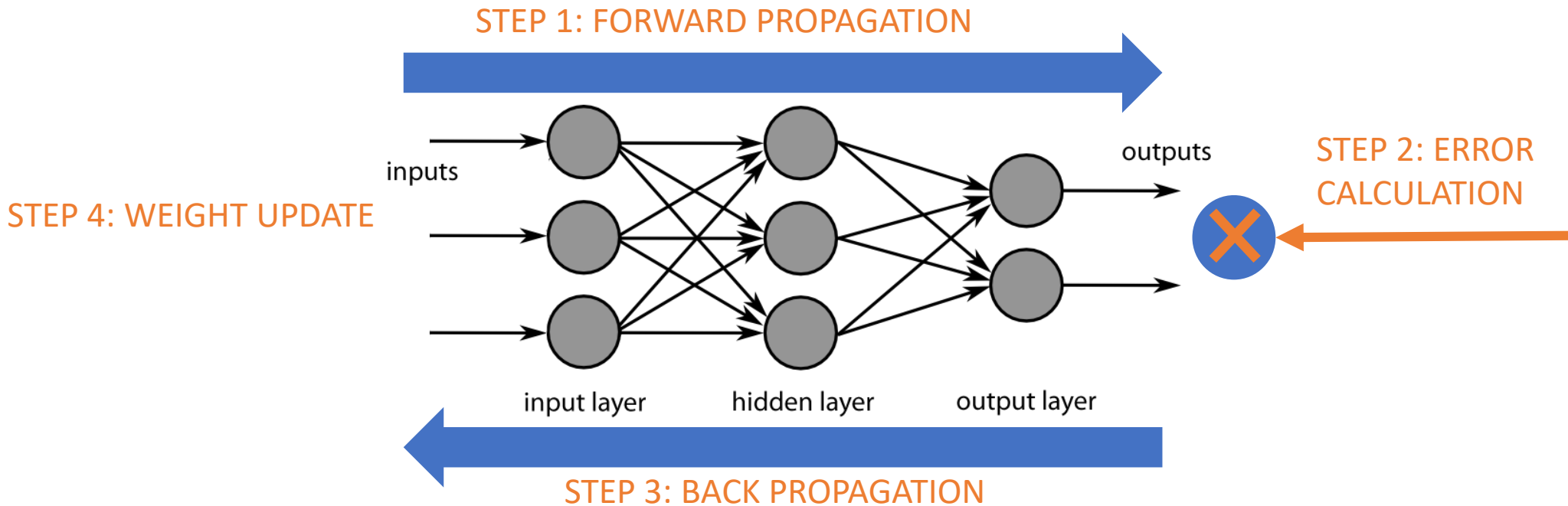
- Backpropagation Phase 1: propagation

- Propagation forward through the network to generate the output value(s)
- Calculation of the cost (error term)
- Propagation of output activations back through network using training pattern target in order to generate the deltas (difference between targeted and actual output values)



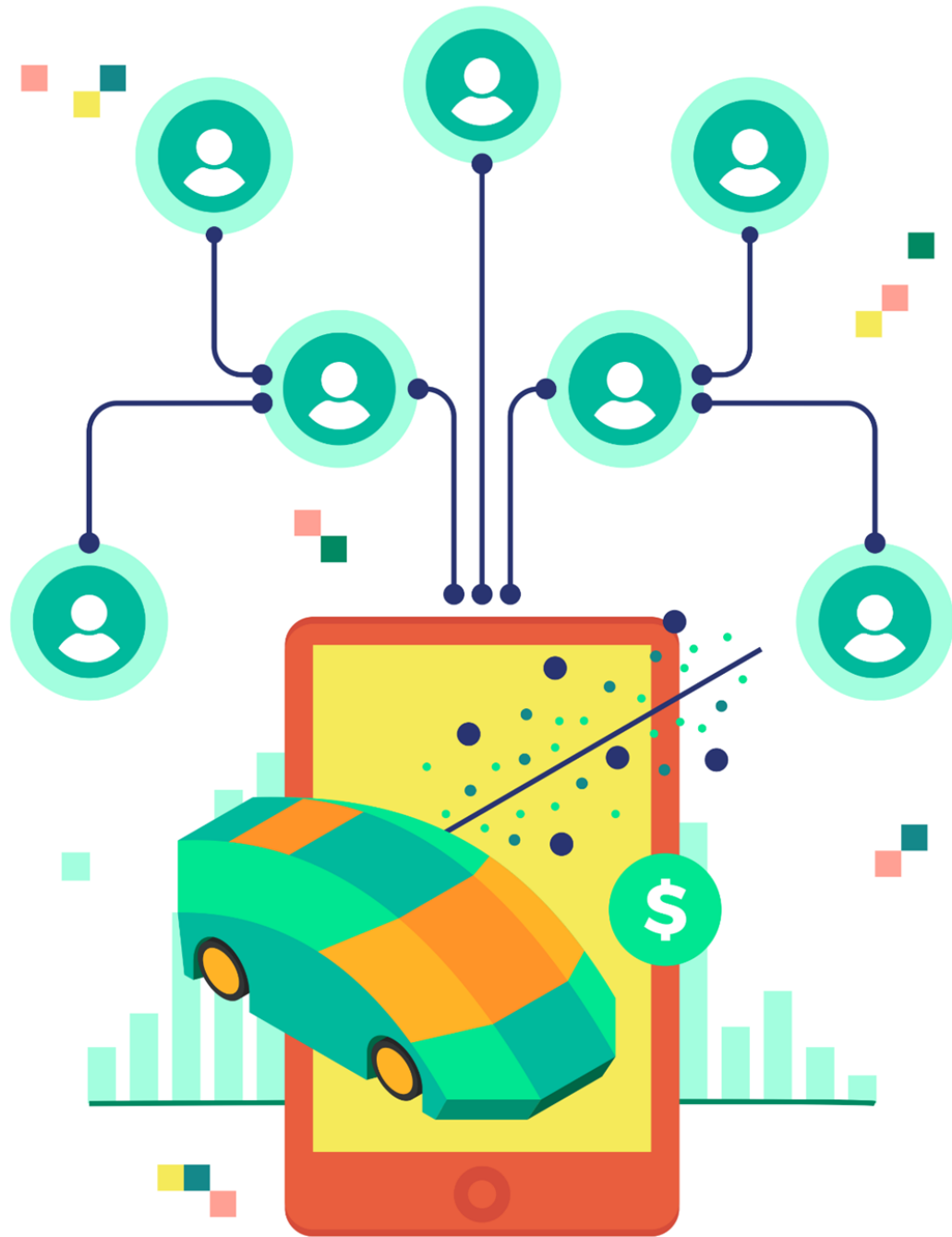
NETWORK TRAINING: BACK PROPAGATION

- Phase 2: weight update
 - Calculate weight gradient.
 - A ratio (percentage) of the weight's gradient is subtracted from the weight.
 - This ratio influences the speed and quality of learning and called learning rate. The greater the ratio, the faster neuron train, but lower ratio, more accurate the training is.



READING MATERIALS: BACK PROPAGATION

- “Backpropagation neural networks: A tutorial” by Barry J.Wythoff
- “Improved backpropagation learning in neural networks with windowed momentum”, International Journal of Neural Systems, vol. 12, no.3&4, pp. 303-318.



MULTI- NEURON MODEL

TWO NEURON MODEL: MATRIX REPRESENTATION

- The network is represented by a matrix of weights, inputs and outputs.
- Total Number of adjustable parameters = 8:
 - Weights = 6
 - Biases = 2

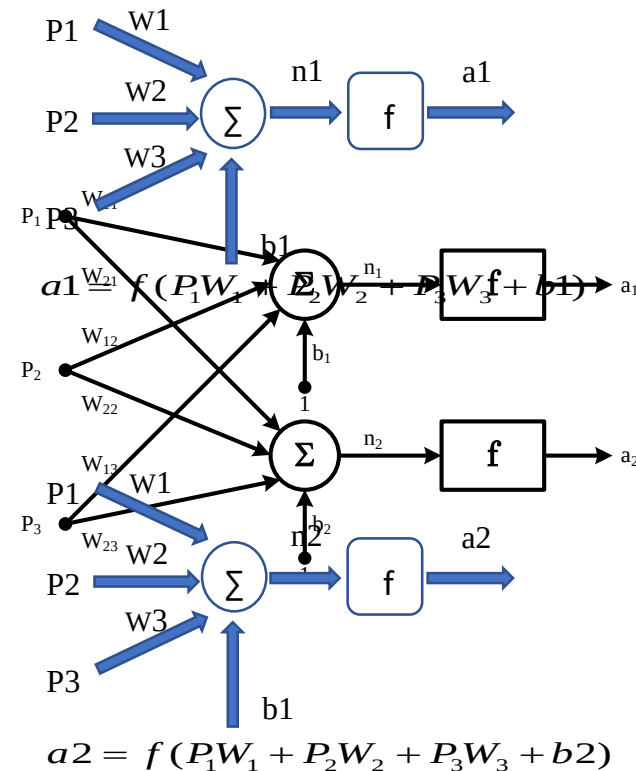
Matrix Representation

$$P = \begin{bmatrix} P_1 \\ P_2 \\ P_3 \end{bmatrix}$$

$$W = \begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \end{bmatrix}$$

$$b = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

$$a = f(W \times P + b)$$



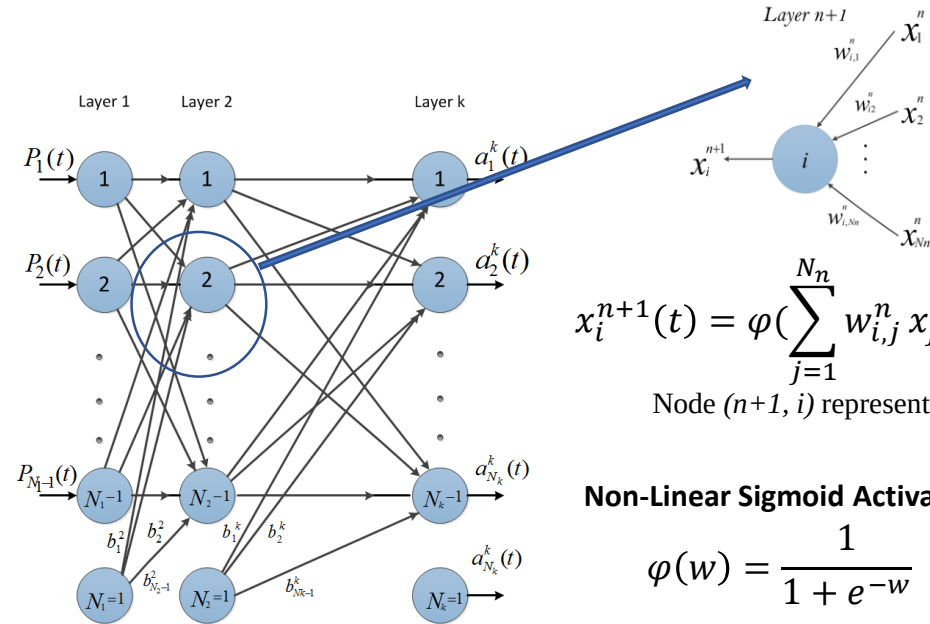
MULTI-LAYER PERCEPTRON MODEL: MATRIX REPRESENTATION

$$P = \begin{bmatrix} P_1 \\ P_2 \\ \vdots \\ P_{N_1} \end{bmatrix}$$

$$\begin{bmatrix} W_{11} & W_{12} & \dots & W_{1,N_1} \\ W_{21} & W_{22} & \dots & W_{2,N_1} \\ \vdots & \vdots & \ddots & \vdots \\ W_{m-1,1} & W_{m-1,2} & \dots & W_{m-1,N_1} \\ W_{m,1} & W_{m,2} & \dots & W_{m,N_1} \end{bmatrix}$$

m : number of neurons in the hidden layer

N_1 : number of inputs



$$x_i^{n+1}(t) = \varphi\left(\sum_{j=1}^{N_n} w_{i,j}^n x_j^n(t)\right)$$

Node $(n+1, i)$ representation

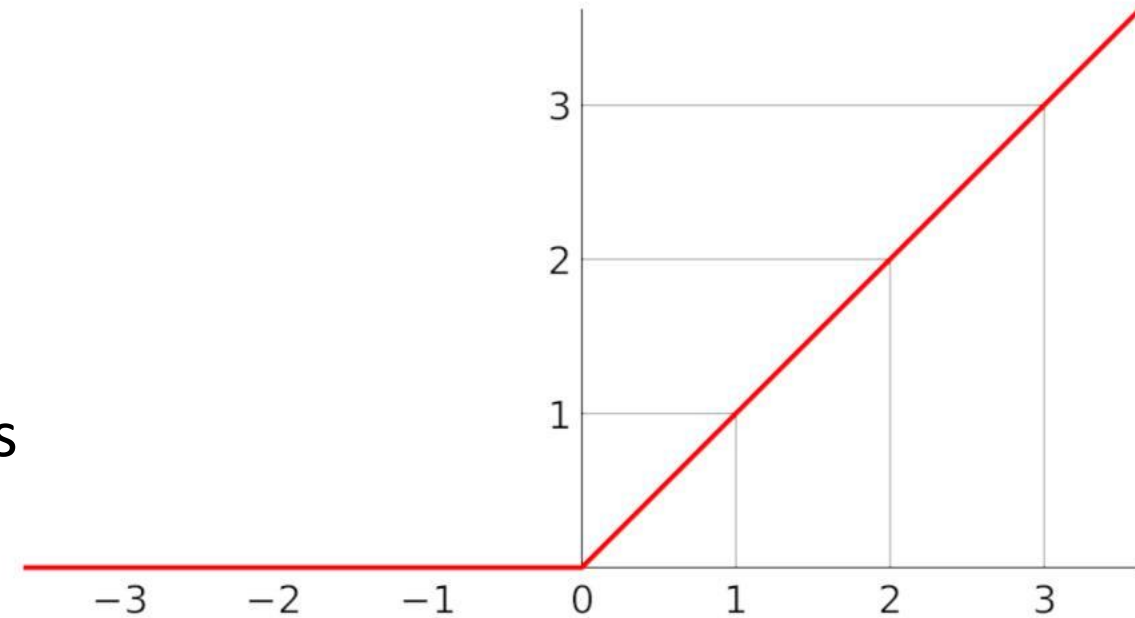
Non-Linear Sigmoid Activation function

$$\varphi(w) = \frac{1}{1 + e^{-w}}$$

ACTIVATION FUNCTIONS

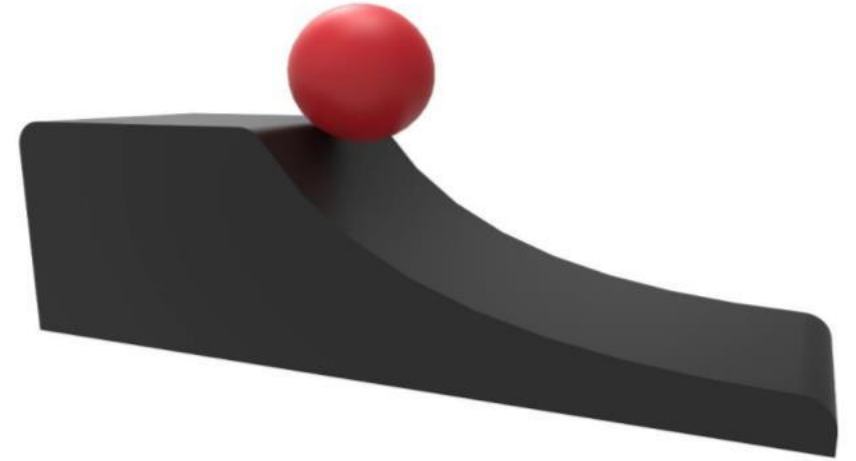
(aka rectifier)

- Step functions don't work with gradient descent – there is no gradient!
 - Mathematically, they have no useful derivative.
- Alternatives:
 - Logistic function
 - Hyperbolic tangent function
 - Exponential linear unit (ELU)
 - ReLU function (Rectified Linear Unit)
- ReLU is common. Fast to compute and works well.
 - Also: “Leaky ReLU”, “Noisy ReLU”
 - ELU can sometimes lead to faster learning though.



ReLU function

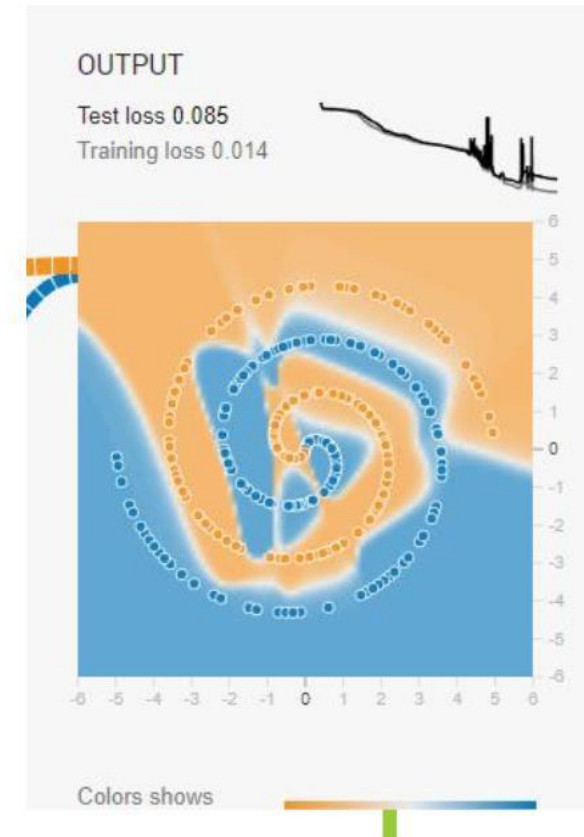
OPTIMIZATION FUNCTIONS



- There are faster (as in faster learning) optimizers than gradient descent
 - Momentum Optimization
 - Introduces a momentum term to the descent, so it slows down as things start to flatten and speeds up as the slope is steep
 - Nesterov Accelerated Gradient
 - A small tweak on momentum optimization – computes momentum based on the gradient slightly ahead of you, not where you are
 - RMSProp
 - Adaptive learning rate to help point toward the minimum
 - Adam
 - Adaptive moment estimation – momentum + RMSProp combined
 - Popular choice today, easy to use

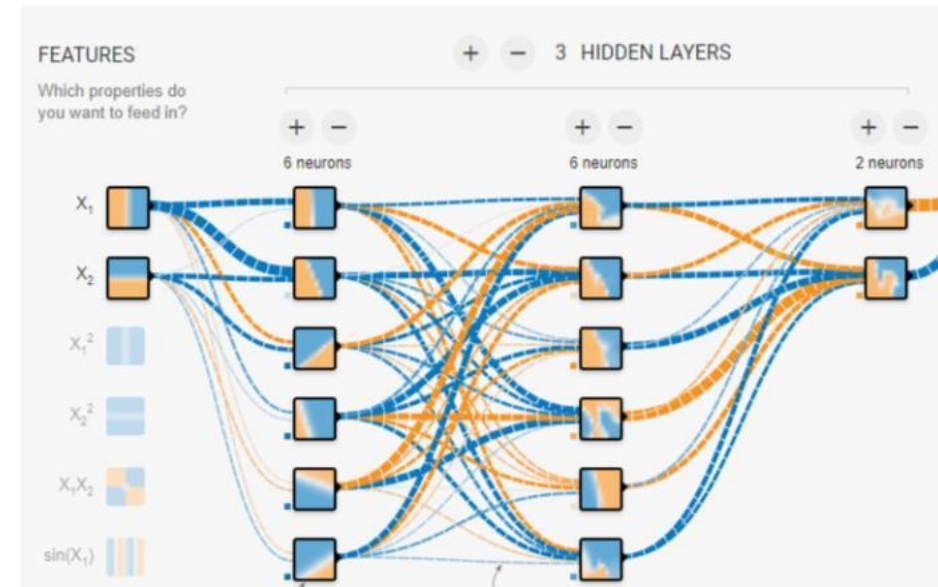
AVOIDING OVERFITTING

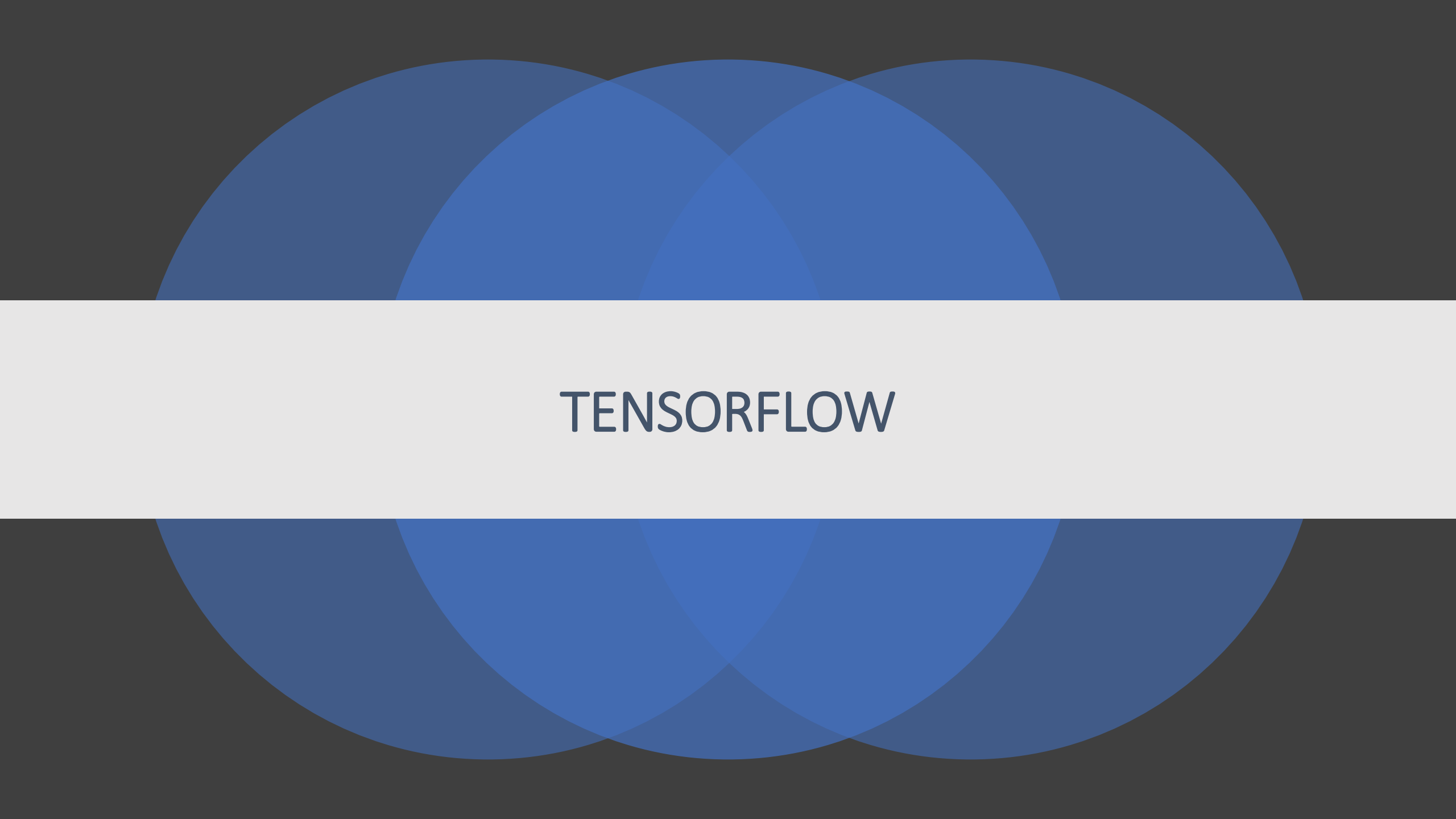
- With thousands of weights to tune, overfitting is a problem
- Early stopping (when performance starts dropping)
- Regularization terms added to cost function during training
- Dropout – ignore say 50% of all neurons randomly at each training step
 - Works surprisingly well!
 - Forces your model to spread out its learning



TUNING YOUR TOPOLOGY

- Trial & error is one way
 - Evaluate a smaller network with less neurons in the hidden layers
 - Evaluate a larger network with more layers
 - Try reducing the size of each layer as you progress – form a funnel
- More layers can yield faster learning
- Or just use more layers and neurons than you need, and don't care because you use early stopping.
- Use “model zoos”



The background of the image features the TensorFlow logo, which consists of three overlapping circles in a triangular arrangement. The top circle is a medium blue, while the two bottom circles are a darker blue. A horizontal white band cuts across the middle of the circles.

TENSORFLOW

WHY TENSORFLOW?

- It's not specifically for neural networks– it's more generally an architecture for executing a graph of numerical operations
- Tensorflow can optimize the processing of that graph, and distribute its processing across a network
 - Sounds a lot like Apache Spark, eh?
- It can also distribute work across GPU's!
 - Can handle massive scale – it was made by Google
- Runs on about anything
- Highly efficient C++ code with easy to use Python API's

TENSORFLOW BASICS

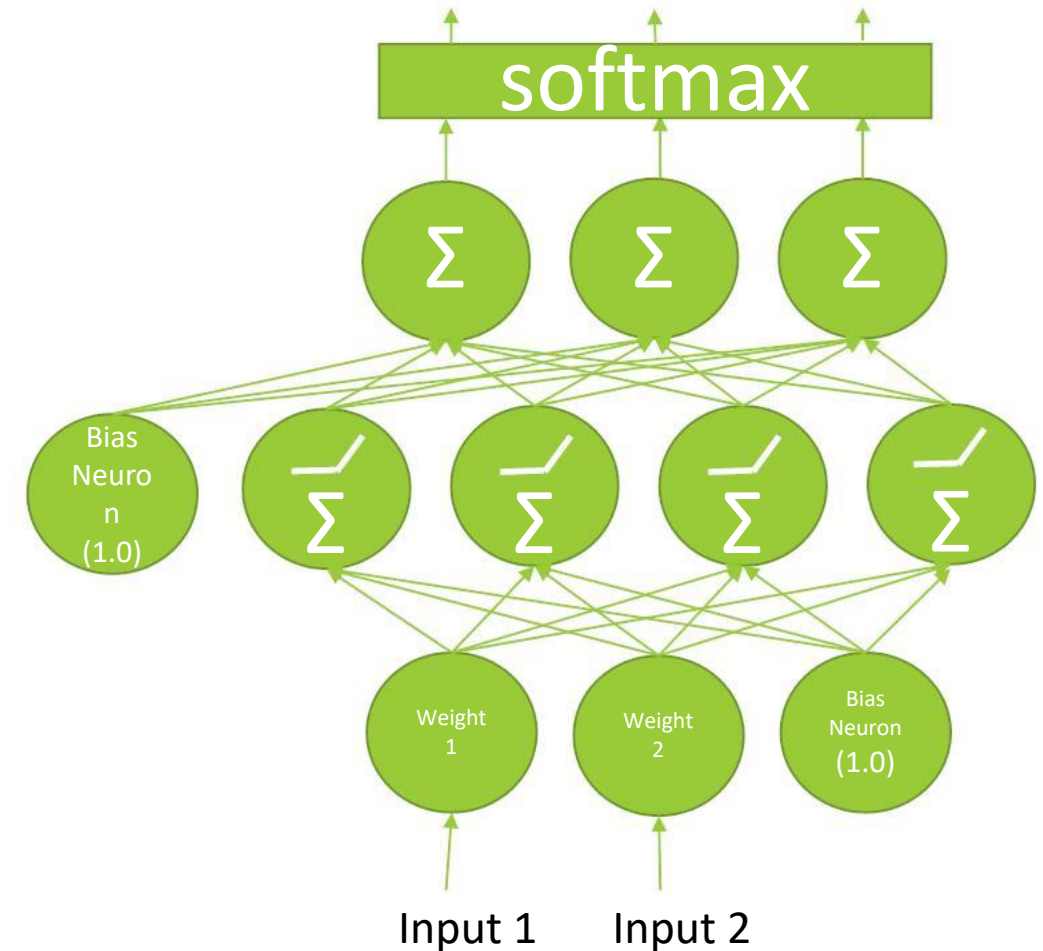
- Install with conda install tensorflow.

Note: This is not easy. Please follow the instructor in the class.

- A tensor is just a fancy name for an array or matrix of values
- To use Tensorflow, you:
 - Construct a graph to compute your tensors
 - Initialize your variables
 - Execute that graph – nothing actually happens until then

CREATING A NEURAL NETWORK WITH TENSORFLOW

- Mathematical insights:
 - All those interconnected arrows multiplying weights can be thought of as a big matrix multiplication
 - The bias term can just be added onto the result of that matrix multiplication
- So in Tensorflow, we can define a layer of a neural network as:
output =
`tf.matmul(previous_layer,
layer_weights) + layer_biases`
- By using Tensorflow directly we're kinda doing this the "hard way."



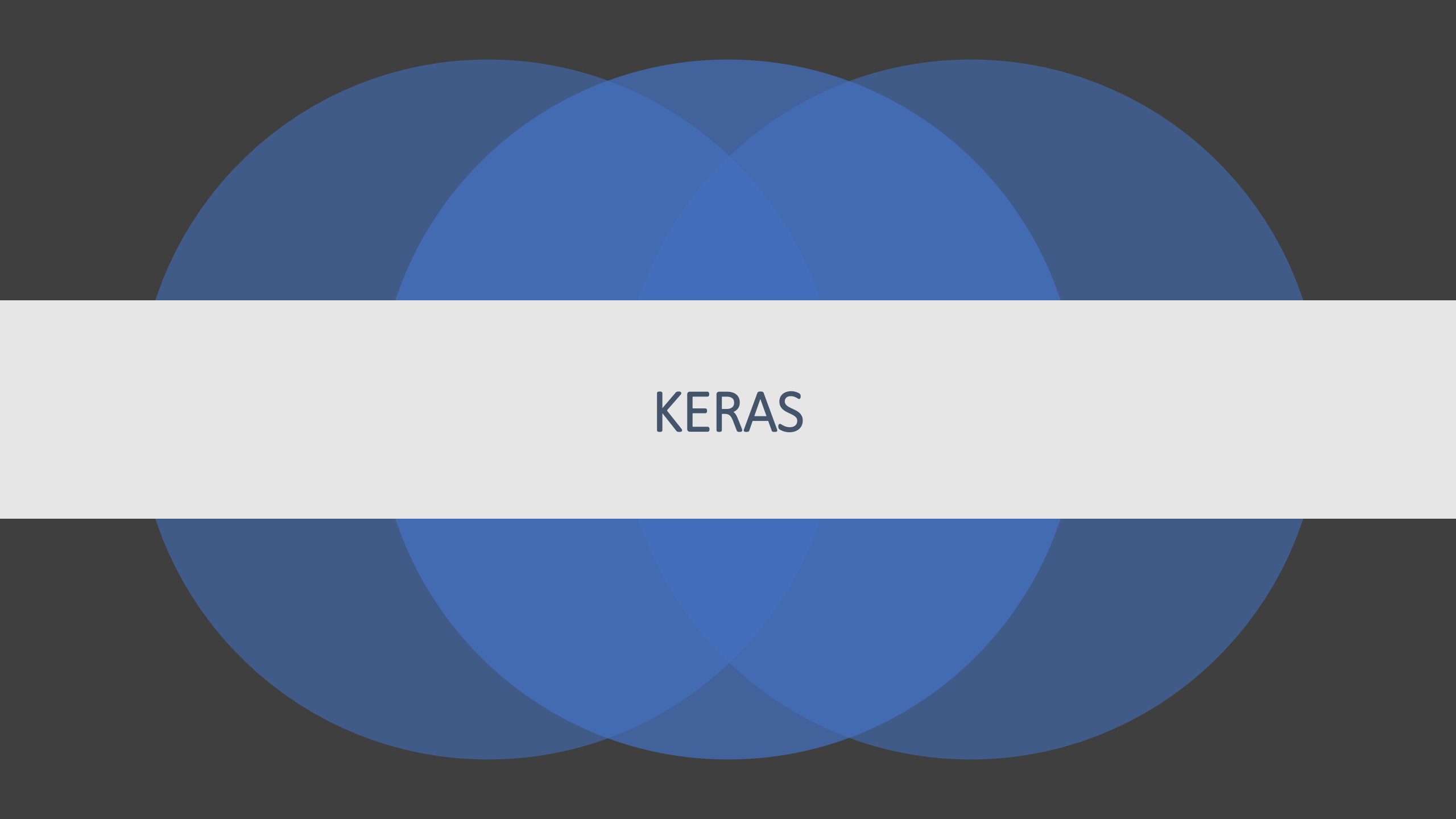
CREATING A NEURAL NETWORK WITH TENSORFLOW

- Load up our training and testing data
- Construct a graph describing our neural network
- Associate an optimizer (ie gradient descent) to the network
- Run the optimizer with your training data
- Evaluate your trained network with your testing data



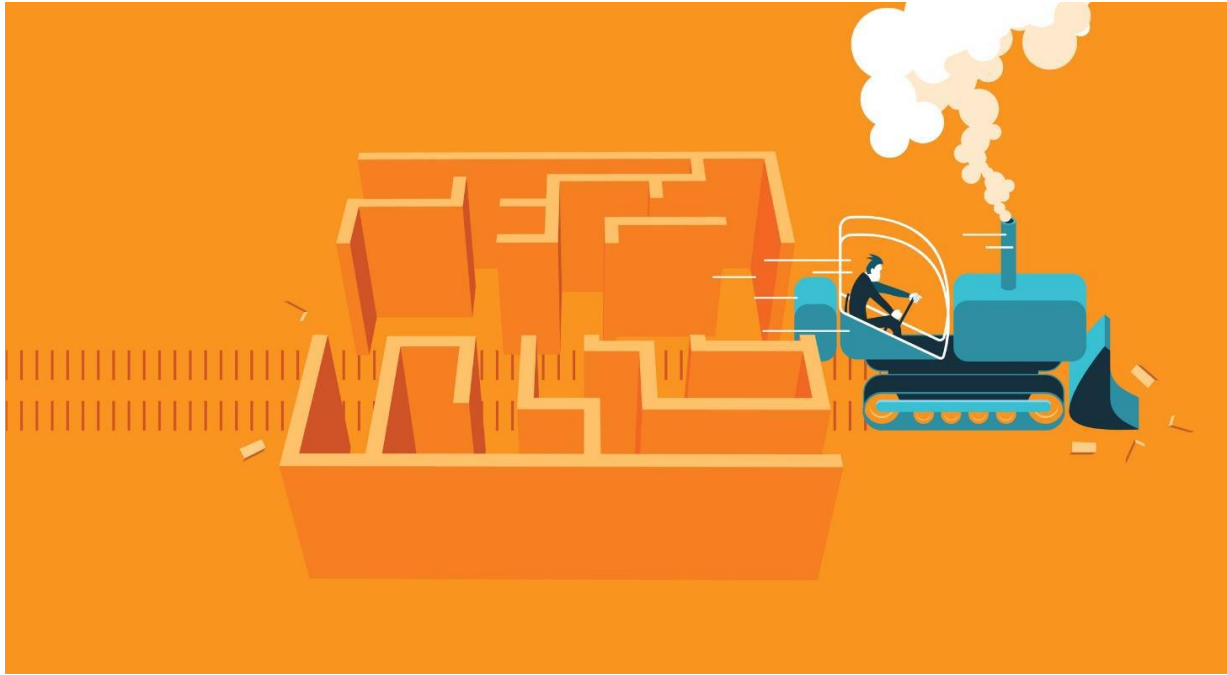
FEATURES ARE NORMALIZED

- Neural networks usually work best if your input data is normalized.
 - That is, 0 mean and unit variance
 - The real goal is that every input feature is comparable in terms of magnitude
- scikit_learn's StandardScaler can do this for you
- Many data sets are normalized to begin with – such as the one we're about to use.

The background of the image features a dark gray field with three overlapping circles in two shades of blue. A horizontal white band cuts across the middle of the circles. The word "KERAS" is centered within this white band.

KERAS

WHY KERAS?



- Easy and fast prototyping
 - Runs on top of TensorFlow (or CNTK, or Theano)
 - scikit_learn integration
 - Less to think about – which often yields better results without even trying
 - This is really important! The faster you can experiment, the better your results.

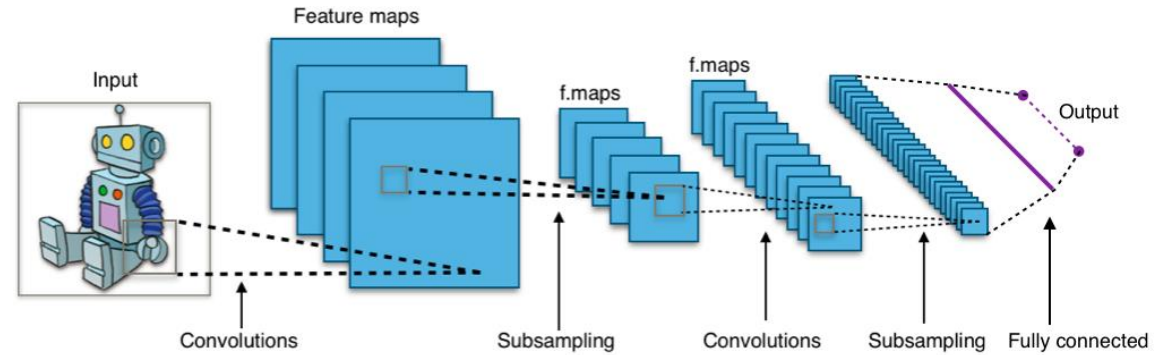
convolutional neural networks

cnn's: what are they for?

- When you have data that doesn't neatly align into columns
 - Images that you want to find features within
 - Machine translation
 - Sentence classification
 - Sentiment analysis
- They can find features that aren't in a specific spot
 - Like a stop sign in a picture
 - Or words within a sentence
- They are “feature-location invariant”



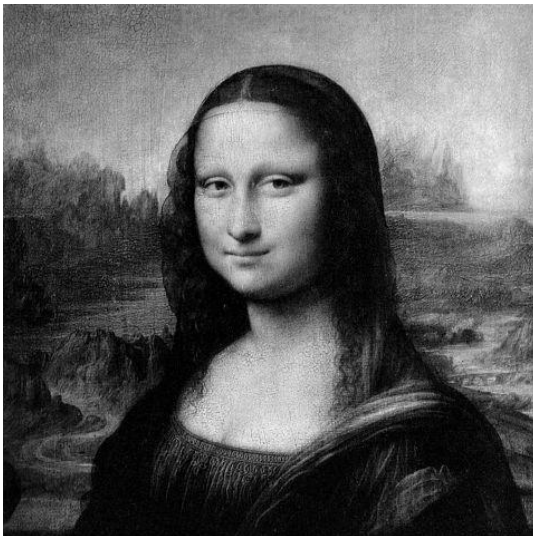
cnn's: how do they work?



- Inspired by the biology of the visual cortex
 - Local receptive fields are groups of neurons that only respond to a part of what your eyes see (subsampling)
 - They overlap each other to cover the entire visual field (convolutions)
 - They feed into higher layers that identify increasingly complex images
 - Some receptive fields identify horizontal lines, lines at different angles, etc. (filters)
 - These would feed into a layer that identifies shapes
 - Which might feed into a layer that identifies objects
- For color images, extra layers for red, green, and blue

HOW TO DIGITALLY REPRESENT AN IMAGE?

- A greyscale image is system of 256 tones with values ranging from 0-255.
- '0' represents black and '255' represents white.
- Numbers in-between represents greys between black and white.
- Binary systems use digits '0' and '1' where '00000000' for black, to '11111111' for white (8-bit image).
- Note: binary value of '11111111' is equal to decimal value of '255'.



255	255	255
155	155	155
0	0	0

11111111	11111111	11111111
10011011	10011011	10011011
00000000	00000000	00000000

how do we “know” that’s a stop sign?

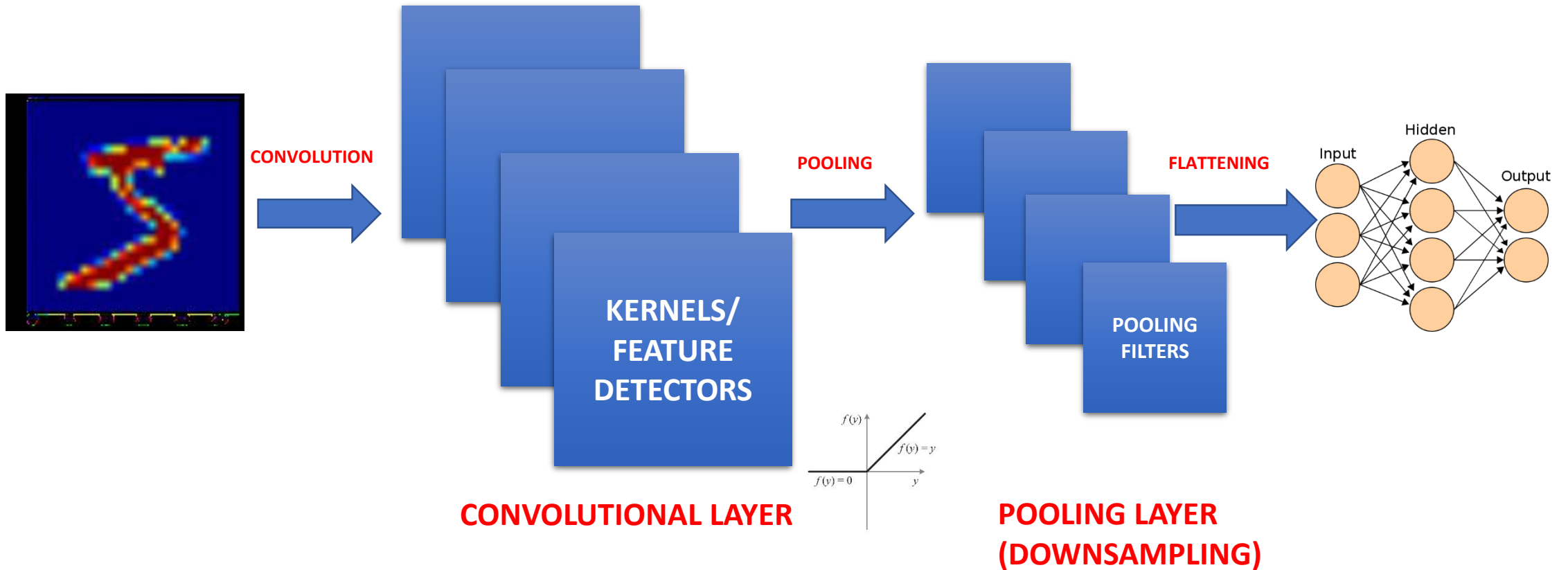
- Individual local receptive fields scan the image looking for edges, and pick up the edges of the stop sign in a layer
- Those edges in turn get picked up by a higher level convolution that identifies the stop sign’s shape (and letters, too)
- This shape then gets matched against your pattern of what a stop sign looks like, also using the strong red signal coming from your red layers
- That information keeps getting processed upward until your foot hits the brake!
- A CNN works the same way



cnn's with keras

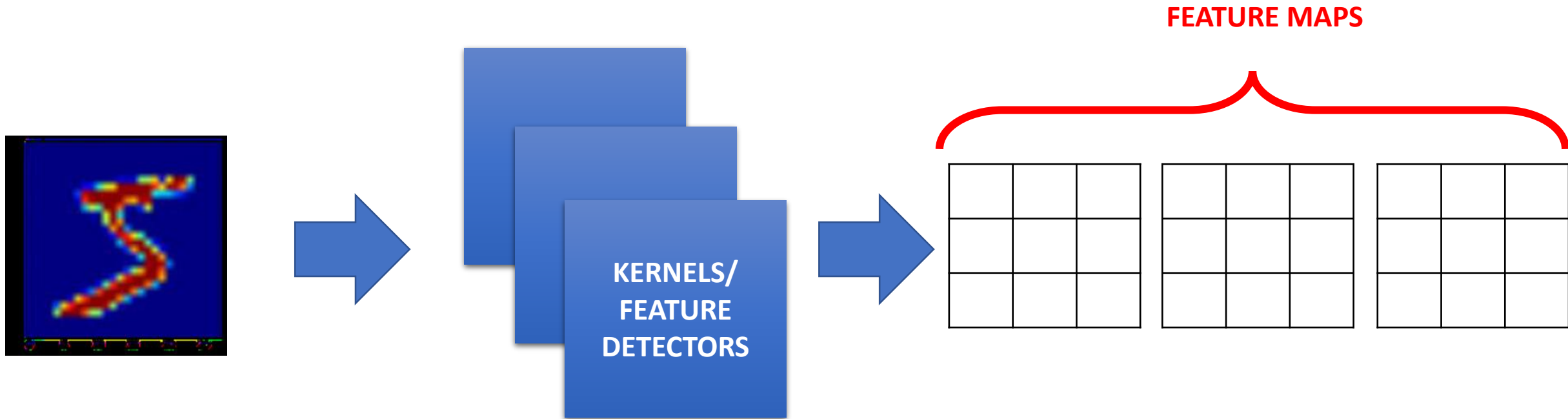
- Source data must be of appropriate dimensions
 - ie width x length x color channels
- Conv2D layer type does the actual convolution on a 2D image
 - Conv1D and Conv3D also available – doesn't have to be image data
- MaxPooling2D layers can be used to reduce a 2D layer down by taking the maximum value in a given block
- Flatten layers will convert the 2D layer to a 1D layer for passing into a flat hidden layer of neurons
- Typical usage:
 - Conv2D -> MaxPooling2D -> Dropout -> Flatten -> Dense -> Dropout -> Softmax

CONVOLUTIONAL NEURAL NETWORKS: ENTIRE NETWORK OVERVIEW



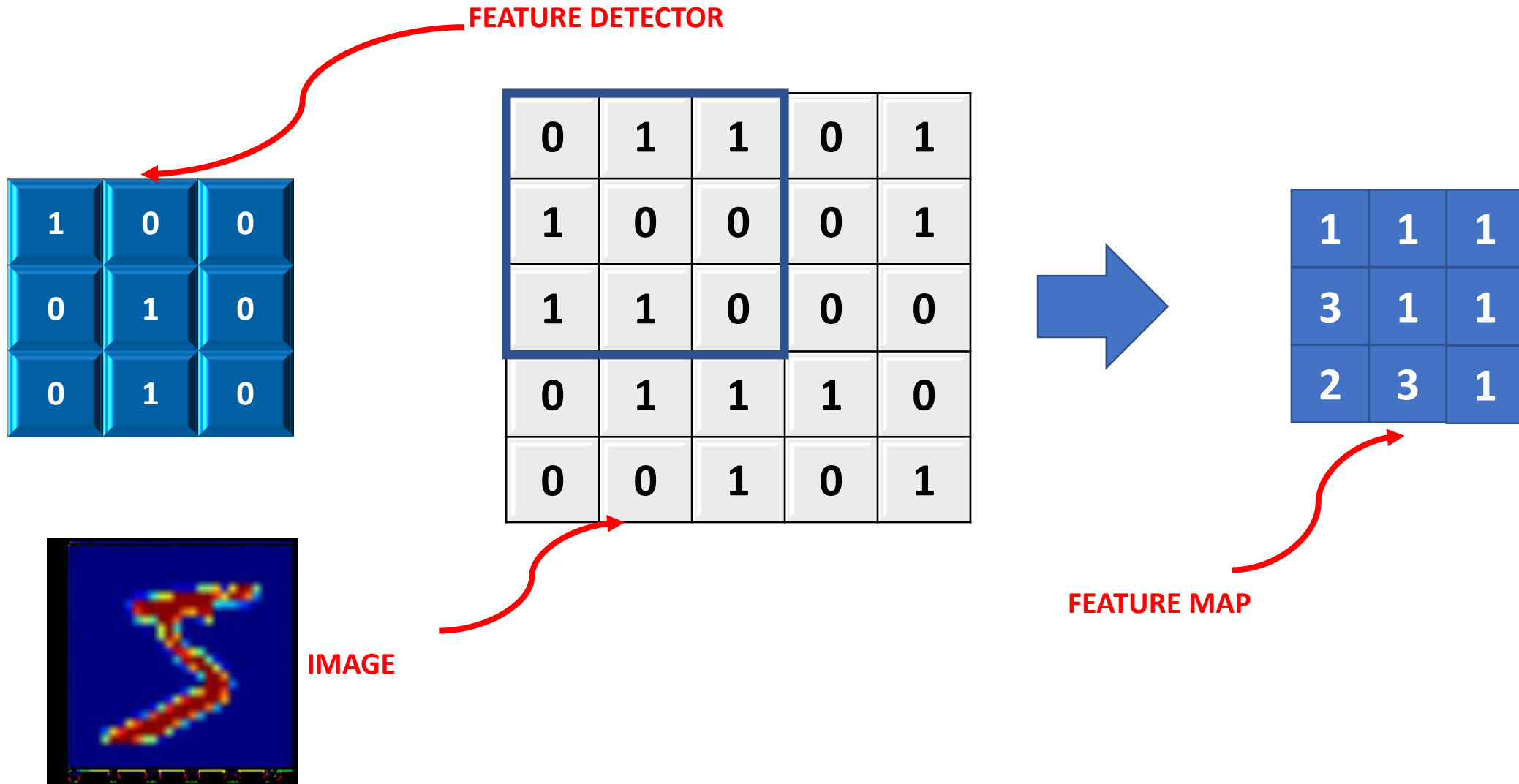
FEATURE DETECTORS

- Convolutions use a kernel matrix to scan a given image and apply a filter to obtain a certain effect.
- An image Kernel is a matrix used to apply effects such as blurring and sharpening.
- Kernels are used in machine learning for feature extraction to select most important pixels of an image.
- Convolution preserves the spatial relationship between pixels.



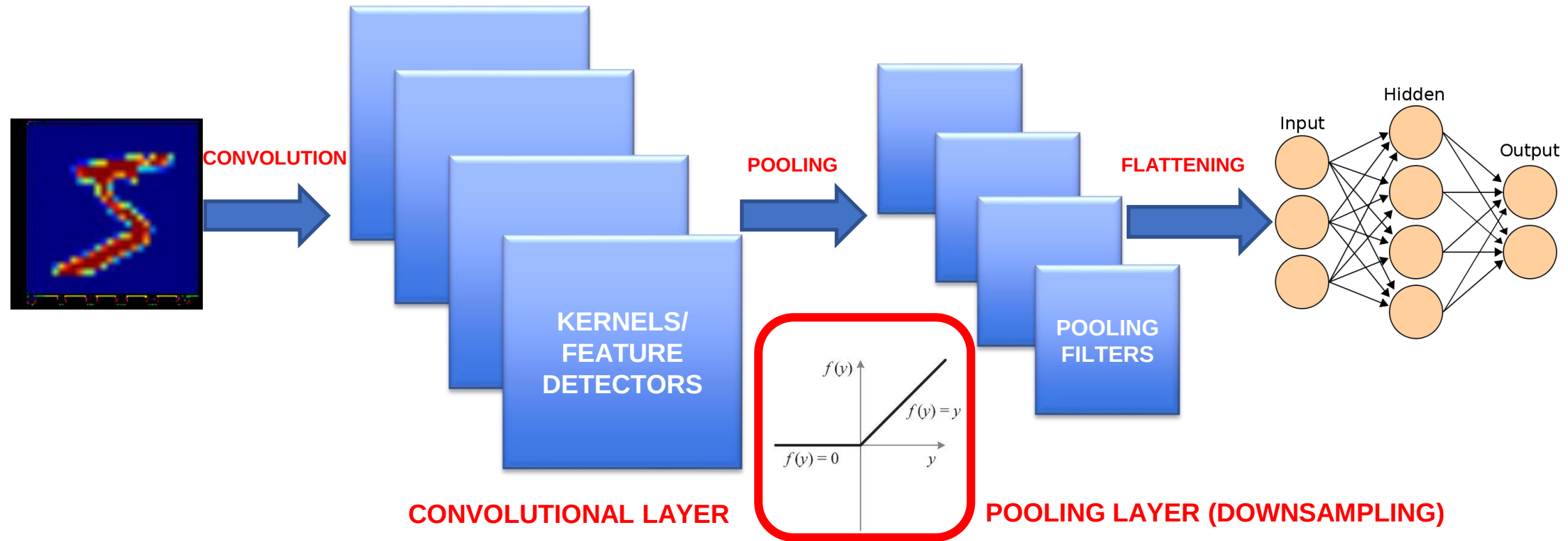
FEATURE DETECTORS

- Live Convolution: <http://setosa.io/ev/image-kernels/>



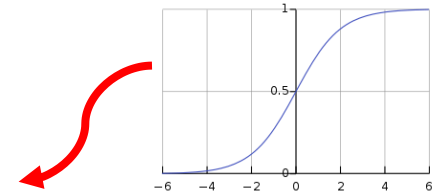
RELU (RECTIFIED LINEAR UNITS)

- RELU Layers are used to add non-linearity in the feature map.
- It also enhances the sparsity or how scattered the feature map is

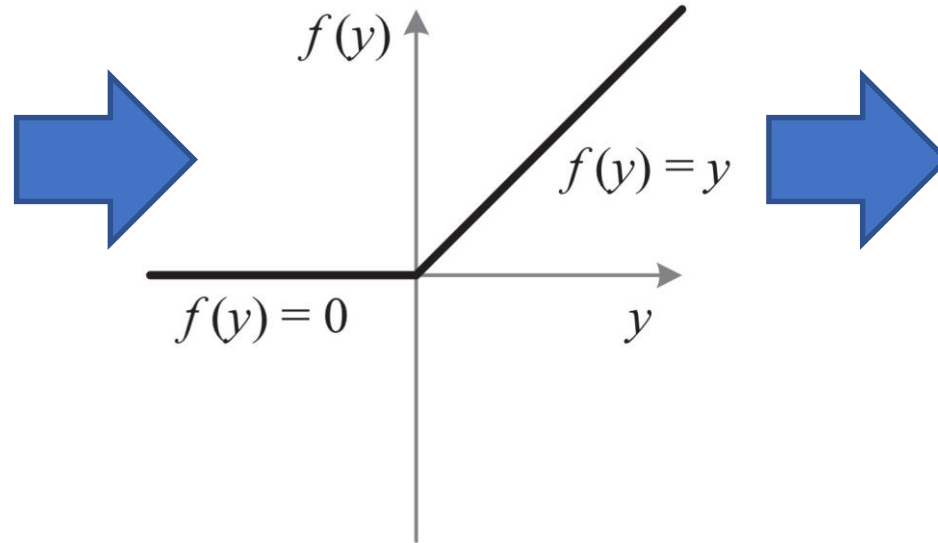


RELU (RECTIFIED LINEAR UNITS)

- RELU Layers are used to add non-linearity in the feature map.
- It also enhances the sparsity or how scattered the feature map is.
- The gradient of the RELU does not vanish as we increase x compared to the sigmoid function



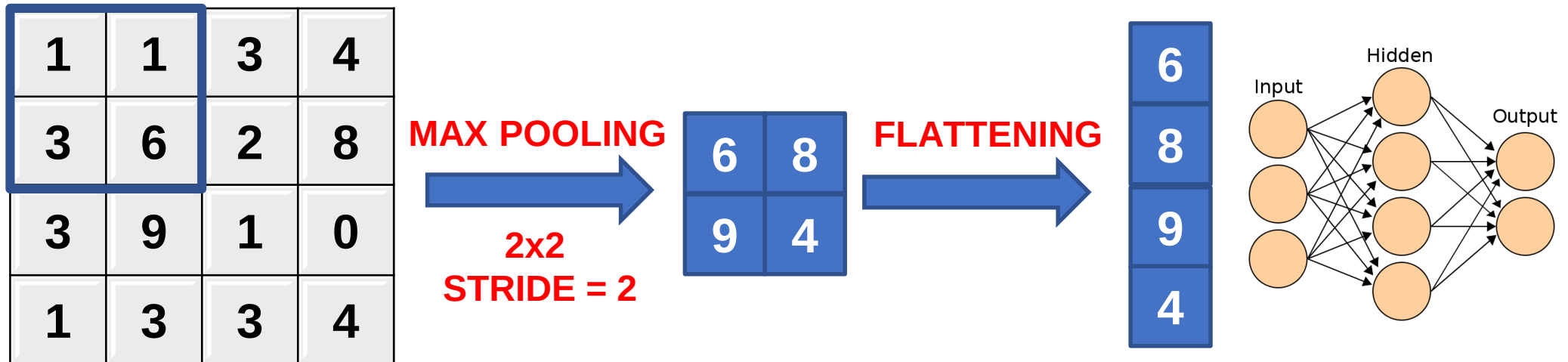
7	10	-5	2	1
1	0	2	3	-6
1	17	-5	0	0
0	1	1	1	0
0	0	-8	12	1



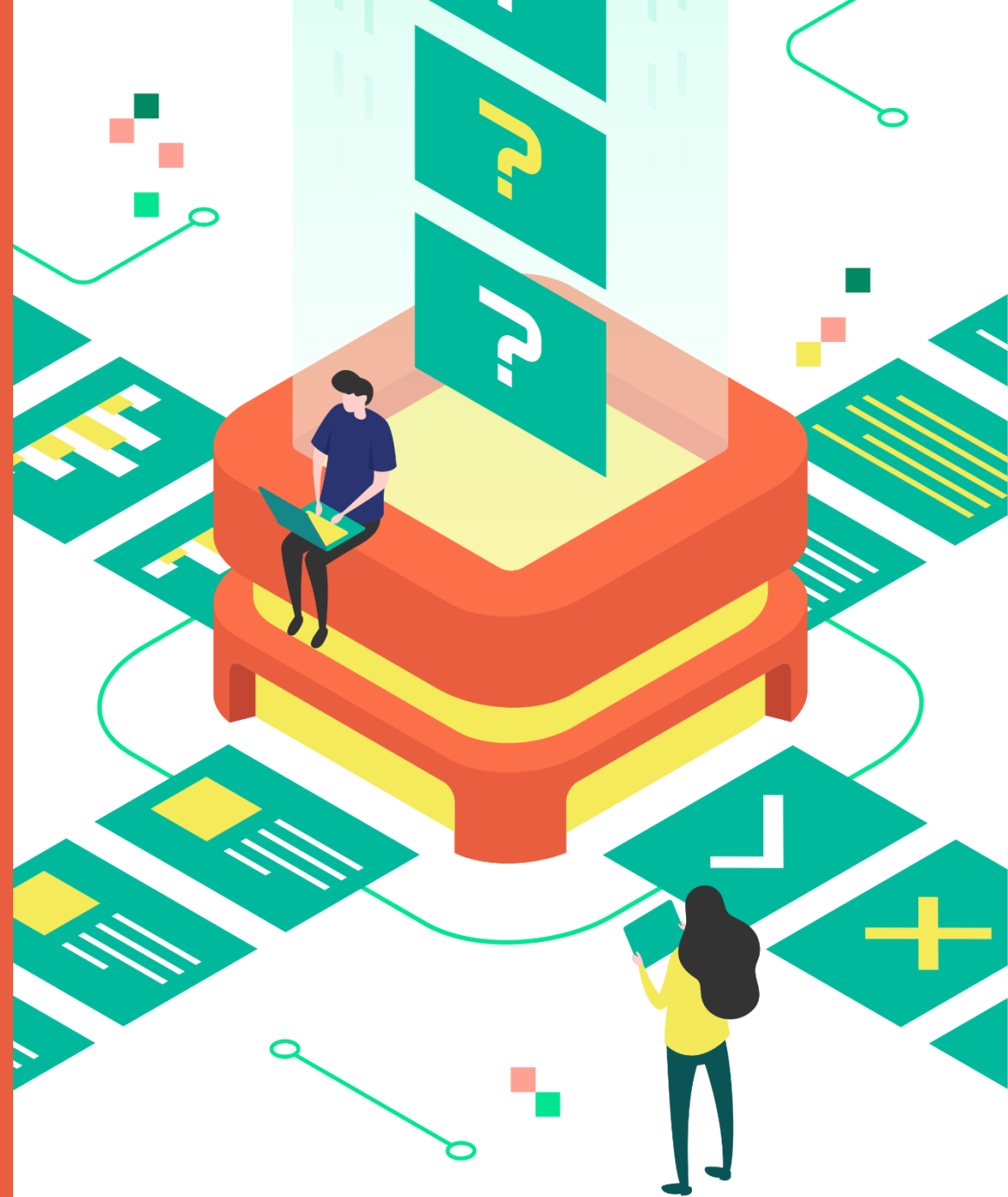
7	10	0	2	1
1	0	2	3	0
1	17	0	0	0
0	1	1	1	0
0	0	0	12	1

POOLING (DOWNSAMPLING)

- Pooling or down sampling layers are placed after convolutional layers to reduce feature map dimensionality.
- This improves the computational efficiency while preserving the features.
- Pooling helps the model to generalize by avoiding overfitting.
- If one of the pixel is shifted, the pooled feature map will still be the same.
- Max pooling works by retaining the maximum feature response within a given sample size in a feature map.
- Live illustration : https://adamharley.com/nn_vis/

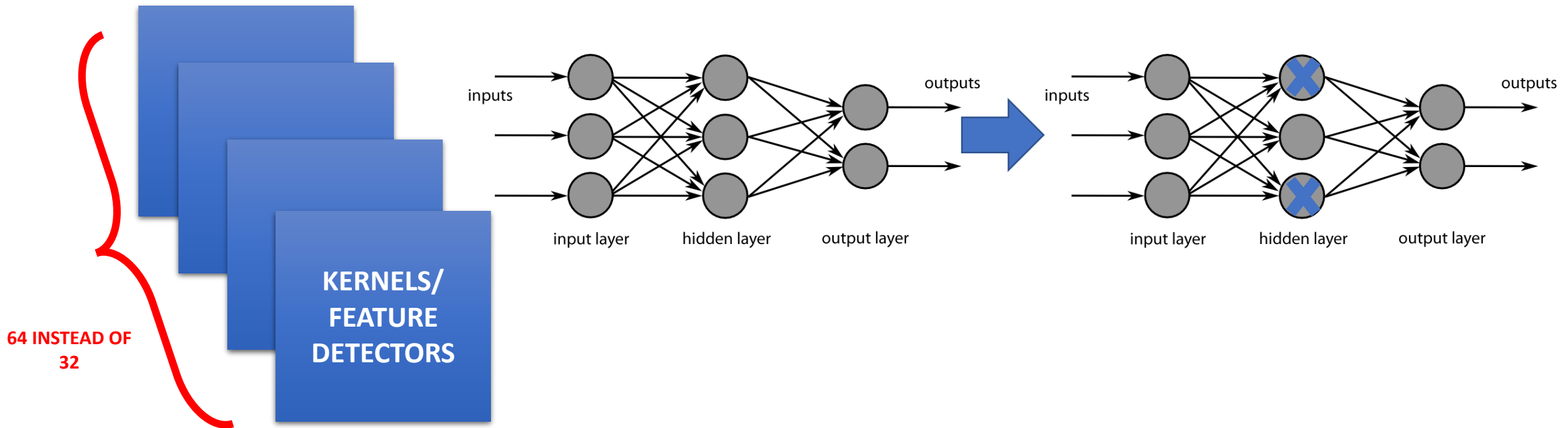


HOW TO IMPROVE NETWORK PERFORMANCE?



INCREASE FILTERS/DROPOUT

- Improve accuracy by adding more feature detectors/filters or adding a dropout.
- Dropout refers to dropping out units in a neural network.
- Neurons develop co-dependency amongst each other during training
- Dropout is a regularization technique for reducing overfitting in neural networks.
- It enables training to occur on several architectures of the neural network



cnn's are hard

- Very resource-intensive (CPU, GPU, and RAM)
- Lots of hyperparameters

Kernel sizes, many layers with different numbers of units, amount of pooling...
in addition to the usual stuff like number of layers, choice of optimizer

- Getting the training data is often the hardest part! (As well as storing and accessing it)

specialized cnn architectures

- Defines specific arrangement of layers, padding, and hyperparameters
- LeNet-5
 - Good for handwriting recognition
- AlexNet
 - Image classification, deeper than LeNet
- GoogLeNet
 - Even deeper, but with better performance
 - Introduces *inception modules* (groups of convolution layers)
- ResNet (Residual Network)
 - Even deeper – maintains performance via *skip connections*.